

UNIVERSAE

“CLANDESTINO: APLICACIÓN ANDROID DE CITAS PARA BARBEROS QUE TRABAJAN DESDE CASA”

PROYECTO INTERMODULAR DE DESARROLLO DE UNA APLICACIÓN

CICLO FORMATIVO DE GRADO SUPERIOR DE DESARROLLO DE APLICACIONES MULTIPLATAFORMA

JHONNATAN OSPINA HENAO

ALFONSO GARCÍA

Curso 2025-2026

ÍNDICE

1. Resumen 5
2. Abstract 6
3. Introducción 7
3.1 Contexto 7
3.2 Problema que se resuelve 7
3.3 Alcance 8
3.4 Análisis de mercado 8
3.4.1 Datos del sector 8
3.4.2 Aplicaciones existentes 8
4. Descripción y objetivos 9
4.1 Descripción del prototipo 9
4.2 Objetivo general 10
4.3 Objetivos específicos 10
5. Contenidos 10
5.1 Mockups 11
5.2 Modelo de datos 12
5.3 Casos de uso 13
5.4 Clases y métodos principales 14
6. Metodología y planificación 15
6.1 Ciclo de vida 15
6.2 Requisitos de partida 16
6.3 Planificación y tareas 16
6.4 Control de versiones y copias de seguridad 17
7. Desarrollo 18
7.1 Tecnologías y versiones 18

7.2 Arquitectura y diagrama de clases 19
7.3 Requisitos funcionales y no funcionales 20
7.4 Concurrencia: el cupo diario 21
7.5 Diario de desarrollo 22
7.6 Organización de carpetas 22
7.7 Pruebas 23
8. Conclusiones y proyección a futuro 23
8.1 Cumplimiento de los objetivos 23
8.2 Limitaciones 24
8.3 Trabajo futuro 24
8.4 Valoración personal 25
9. Bibliografía 26
10. Anexos 30
Anexo I. Manual de instalación 30
I.1 Requisitos 30
I.2 Obtener el proyecto 30
I.3 Base de datos 31
I.4 Fichero de configuración `.env` 31
I.5 Crear las tablas y el usuario de la API 32
I.6 API 32
I.7 App en el emulador 33
I.8 Problemas frecuentes 33
Anexo II. Manual de usuario 34
II.1 Entrar en la aplicación 34
II.2 Cliente 35
II.3 Barbero 36
Anexo III. Documentación técnica de diseño 41

III.1 Descripción de los casos de uso 41
III.2 Trazabilidad 46
III.3 Diagramas de clases 47
III.4 Diagramas de secuencia de «Solicitar cita» 53
III.5 Mockups restantes 59
III.6 Estados de una cita 61
III.7 Tablas de detalle 62
Anexo IV. Casos de prueba manuales 65
IV.1 Recorrido de la app en el emulador 65
IV.2 Comprobaciones de la API con `curl` 66
IV.3 Cita a mano en el emulador 67
Anexo V. Diario de desarrollo 68
Inicio y definición del alcance · 30 de septiembre y 1 de octubre de 2026 68
Base de datos y diseño · 1 de octubre de 2026 68
Convenciones y contraseñas · 9 de octubre de 2026 69
API y reservas simultáneas · hasta el 14 de octubre de 2026 69
App Android y recorrido completo · hasta el 15 de octubre de 2026 69
Memoria y preparación de la defensa · desde el 16 de octubre de 2026 70
Cita creada por el barbero · 17 de octubre de 2026 70
Balance del trabajo 71

1. RESUMEN

Lo que quiero reflejar en este proyecto es cómo he buscado la solución a un problema de mi propio trabajo. Corto el pelo en casa y no encontré ninguna aplicación de citas que me sirviera, porque las del mercado están pensadas para negocios con local a pie de calle y las más conocidas, como Booksy, funcionan como un escaparate en el que mis clientes verían también otras barberías. Clandestino es la app que he desarrollado para mi propio uso y para otros barberos que trabajan desde casa y quieren fidelizar a su clientela sin mandarla a un marketplace. El acceso es solo por invitación, el barbero decide el cupo diario y la interfaz se queda en lo justo. Es una app nativa de Android en Java, con una API REST propia y una base de datos MySQL, que reconstruye el núcleo de la versión web con la que ya me organizaba. El análisis de siete aplicaciones de reserva muestra que ninguna combina el acceso cerrado, el cupo configurable y la ausencia de cuotas o comisiones. He seguido un ciclo incremental con prácticas de Scrum. En el diseño elaboré el modelo entidad-relación de cinco tablas, diez casos de uso, treinta requisitos funcionales, los mockups y los diagramas de clases y de secuencia. Programé la API sin frameworks, sobre el servidor HTTP del JDK, con contraseñas protegidas mediante PBKDF2, tokens JWT firmados con HMAC, límite de intentos de acceso y consultas parametrizadas. La app sigue el patrón MVVM, hace las peticiones de red en hilos secundarios y guarda el token cifrado con una clave del Android Keystore. Lo más delicado era evitar que dos clientes que piden cita a la vez superen el cupo del día. La base de datos impide dos citas activas a la misma hora con una columna generada e índice único, pero ningún índice cuenta las citas de un día, así que el servicio de citas protege esa operación con un bloqueo por fecha y una única transacción. Una prueba con cinco hilos lo demuestra, porque sin el bloqueo entran cinco citas en un día con cupo de una y con él solo una. El prototipo supera 97 pruebas automáticas en la API, 26 en la app y 21 sobre el esquema, y he recorrido el flujo completo de cliente y barbero en el emulador. También hice la primera función de la segunda fase, la cita que el barbero crea a mano para quien llama por teléfono. Quedan como trabajo futuro el HTTPS y los avisos al cliente.

Palabras clave: Android, API REST, gestión de citas, concurrencia, MySQL.

2. ABSTRACT

What I want to show in this project is how I looked for a solution to a problem in my own work. I cut hair at home and could not find a booking app that worked for me, because the apps on the market are built for businesses with a high-street shop and the best known ones, such as Booksy, work as a marketplace where my clients would also see other barbershops. Clandestino is the app I have built for my own use and for other barbers who work from home and want to keep their own clients loyal instead of sending them to a marketplace. Access is by invitation only, the barber sets the daily limit and the interface keeps to the essentials. It is a native Android app in Java, backed by a custom REST API and a MySQL database, that rebuilds the core of the web version I already used. A review of seven booking apps shows that none of them combines invitation-only access, a configurable daily limit and no fees or commissions. I followed an incremental life cycle with Scrum practices. The design stage produced a five-table entity-relationship model, ten use cases, thirty functional requirements, screen mockups, and class and sequence diagrams. I wrote the API without frameworks, on top of the HTTP server included in the JDK, with PBKDF2 password hashing, HMAC-signed JWT tokens, login attempt limiting and parameterised queries. The app follows the MVVM pattern, runs every network request on a background thread and stores the session token encrypted with an Android Keystore key. The hardest part was making sure that two clients booking at the same time cannot exceed the daily limit. The database blocks two active appointments at the same time slot through a generated column with a unique index, but no index can count the appointments of a day, so the appointment service protects that operation with a per-date lock and a single transaction. A test with five threads proves it, since without the lock five appointments are created on a day limited to one and with it only one. The prototype passes 97 automated tests in the API, 26 in the app and 21 on the database schema, and I have run the full client and barber flow on the Android emulator. I also completed one feature from the second phase, which lets the barber add an appointment by hand for clients who phone in. HTTPS and client notifications are left as future work.

Keywords: Android, REST API, appointment scheduling, concurrency, MySQL.

3. INTRODUCCIÓN

3.1 Contexto

He buscado la manera de poder organizar mis citas de la manera que no sea pagando una suscripción por un saas que tampoco termine de cuadrarse a lo que yo necesito ,ya que casi siempre son clientes que ya conozco o que vienen recomendados, y quería darles la comodidad de ellos poder escoger su hora y su día de la manera que mejor les venga . No tengo local a pie de calle ni lo quiero, y como yo hay muchos barberos que empiezan así o que han decidido quedarse trabajando desde casa.

Para organizarme ya uso una aplicación web propia, también llamada Clandestino, hecha con Next.js y Supabase , desplegada en Vercel . En este proyecto he reconstruido su núcleo como app nativa de Android en Java con vistas XML, con una API REST propia y una base de datos MySQL local. Lo que en la web resuelve Supabase aquí lo he programado yo, y así he aplicado lo aprendido en los módulos del ciclo de Desarrollo de Aplicaciones Multiplataforma (Real Decreto 450/2010, 2010), que la tabla III.14 del anexo III relaciona con cada parte del proyecto.

3.2 Problema que se resuelve

Cuando busqué una aplicación para llevar mis citas, todas daban por hecho un negocio a pie de calle, con su dirección pública y a menudo varios empleados. Las más conocidas, como Booksy, son además un marketplace, así que el cliente entra para reservar contigo y la propia app le enseña otras barberías de la zona. A quien trabaja en casa eso no le interesa, porque vive de una clientela pequeña y fiel que necesita cuidar, no poner delante de la competencia.

Ese barbero quiere decidir quién reserva, porque no va a abrir su puerta a cualquiera, limitar cuántas citas acepta cada día y no tener nunca dos a la misma hora, porque no hay un segundo sillón que absorba el error. Con mensajes sueltos por WhatsApp todo eso depende de su memoria.

Clandestino es la app que me habría gustado encontrar. Solo entran los clientes que el barbero invita y aprueba, él fija el cupo de cada día y el sistema impide la doble

reserva aunque dos clientes pidan a la vez (apartado 7.4). Lo demás lo he dejado fuera para que la app sea simple.

3.3 Alcance

El prototipo cubre el ciclo completo de una cita. El barbero invita al cliente con un código y lo aprueba o lo bloquea, y el cliente pide cita en un hueco libre. El barbero la confirma (a esa hora o a otra) o la rechaza, marca después si el cliente vino y configura su horario semanal, el cupo de cada día, las vacaciones y los días sueltos de apertura.

Quedan fuera las métricas de ingresos, las nóminas, los gastos, los avisos automáticos y el cobro en la app. El prototipo funciona en local, con la API y la base de datos en mi ordenador y la app en el emulador, y publicarlo exigiría cifrar el tráfico con HTTPS.

3.4 Análisis de mercado

3.4.1 Datos del sector

Según el Instituto Nacional de Estadística (INE), a 1 de enero de 2025 había en España 155.724 empresas de «otros servicios personales» (división 96 de la CNAE 2009, que incluye la peluquería). El 60,2 % no tiene asalariados y el 84,7 % son personas físicas (INE, s. f.), así que predomina el negocio de una sola persona, como el del barbero que trabaja en casa. Además, el 92,5 % de las personas de 16 a 74 años usa Internet a diario (INE, 2025) y el 94,6 % de los internautas usa WhatsApp (Comisión Nacional de los Mercados y la Competencia [CNMC], 2026), lo que justifica una app pensada para el móvil.

3.4.2 Aplicaciones existentes

He revisado siete aplicaciones de reserva en sus webs oficiales (tabla 1). Las cuatro primeras están especializadas en belleza y barbería, y las tres últimas sirven para cualquier negocio de servicios.

Ninguna combina el acceso por invitación, el cupo diario y la ausencia de cuotas y comisiones. Las especializadas viven de un escaparate que enseña la competencia a tus propios clientes, y las generalistas piden configurar mucho de lo que un barbero

solo no necesita. Clandestino es para quien trabaja en casa y quiere ordenar y conservar la clientela que ya tiene, no para un salón con empleados.

Tabla 1

Comparativa de aplicaciones de reserva de citas

Aplicación	Precio (web oficial, 30-09-2026)	Por qué no encaja
Booksy	34,99 € + IVA al mes y comisiones	Enseña otras barberías a tus clientes
Fresha	19,95 € al mes y comisiones	Escaparate para captar clientes
Treatwell	25 % de cada cliente nuevo	Vive de traer clientes nuevos
AgendaPro	Desde 10 € al mes	Personal, inventario y caja
Square	Gratis, 19 € o 49 € al mes	Vive de su pasarela de pago
Setmore	Gratis o 5 $ por usuario y mes	Sin aprobación de clientes
Calendly	Gratis o 10 $ por usuario y mes	Sin cupo diario ni aprobaciones

Nota. Elaboración propia a partir de las webs oficiales de Booksy (s. f.), Fresha (s. f.), Treatwell (s. f.), AgendaPro (s. f.), Square (s. f.), Setmore (s. f.) y Calendly (s. f.).

4. DESCRIPCIÓN Y OBJETIVOS

4.1 Descripción del prototipo

El prototipo tiene tres piezas que se comunican por la red local del ordenador de desarrollo. La **app Android**, en Java con vistas XML y Material 3, sigue el patrón MVVM, funciona desde Android 8.0 (API 26) y es lo único que ven los usuarios. La **API REST**, en Java 21 sobre el servidor HTTP del JDK y sin frameworks, recibe peticiones en JSON, aplica las reglas del negocio y es la única que entra en la **base de datos MySQL 9.7**, que guarda usuarios, invitaciones, horarios, excepciones de calendario y citas en cinco tablas.

La app reconoce dos tipos de usuario y muestra a cada uno su panel con una barra de navegación inferior. El **cliente** entra con su @usuario y su contraseña, y mientras no está aprobado ve una pantalla de espera. Después tiene tres pestañas. En «Mis citas» ve sus citas con su estado y cancela las que siguen activas, en «Pedir cita» elige un día de las dos próximas semanas y una hora libre, y en «Ajustes» cambia el tema y cierra la sesión.

El **barbero** tiene cuatro pestañas. En «Solicitudes» confirma cada cita pendiente con la hora pedida o con otra, o la rechaza. En «Agenda» ve las citas confirmadas por semanas, marca si el cliente vino y mete a mano la cita de quien llama por teléfono. En «Horarios» fija el tramo y el cupo de cada día y cierra días por

vacaciones, y en «Clientes» genera códigos de invitación y aprueba o bloquea a cada cliente.

Solo las citas pendientes y confirmadas ocupan hueco y cuentan para el cupo, así que cancelar o rechazar una cita libera su hora en ese momento. El diagrama de estados completo está en la figura III.19 del anexo III.

4.2 Objetivo general

Desarrollar una aplicación Android, con su API y su base de datos, que permita a un barbero que trabaja desde casa gestionar las citas de su propia clientela con acceso por invitación, cupo diario y sin riesgo de reservas duplicadas, de forma más sencilla que las plataformas del mercado.

4.3 Objetivos específicos

1. Diseñar una base de datos relacional en MySQL que impida por sí misma dos citas activas a la misma hora del mismo día.
2. Programar una API REST en Java, sin frameworks, que autentique a los usuarios con contraseñas protegidas y tokens firmados y que separe lo que puede hacer cada rol.
3. Asegurar que el cupo diario se respeta aunque varios clientes pidan cita a la vez, y demostrarlo con una prueba automática de hilos concurrentes.
4. Construir una app Android en Java con arquitectura MVVM que cubra las pantallas de cliente y barbero sin bloquear el hilo principal con la red.
5. Comprobar el funcionamiento con pruebas unitarias y de integración y con un recorrido completo en el emulador, documentado con capturas.

5. CONTENIDOS

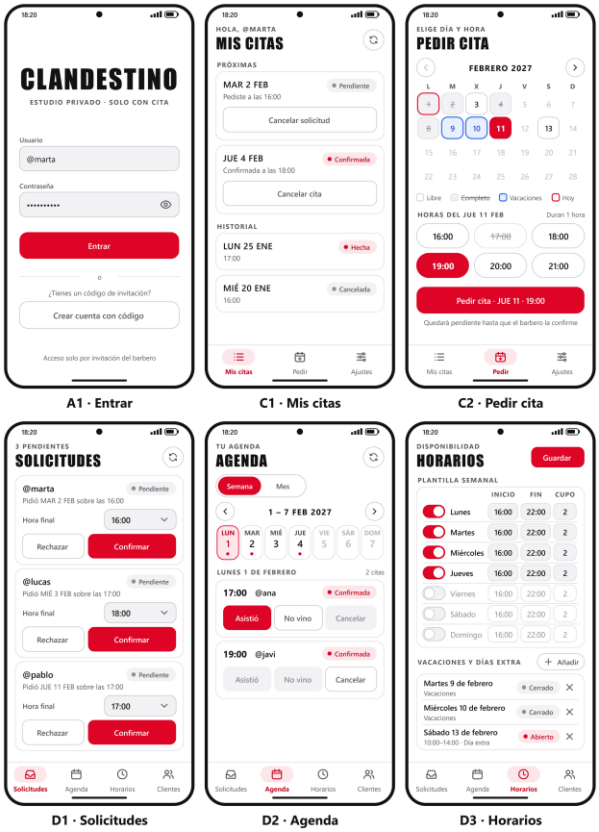
Este apartado recoge el diseño previo al código: cómo se iban a ver las pantallas, cómo se guardan los datos, qué puede hacer cada usuario y qué clases sostienen la lógica. Dibujé los diagramas en draw.io y los actualicé al terminar para que coincidan con lo programado.

5.1 Mockups

Antes de escribir los layouts dibujé once mockups, uno por pantalla y variante. La figura 1 reúne los seis principales, y los otros cinco están en el anexo III junto con la comparación entre el mockup de «Pedir cita» y la app terminada (figuras III.17 y III.18).

Figura 1

Mockups de las pantallas principales: entrada, mis citas y pedir cita del cliente; solicitudes, agenda y horarios del barbero



Nota. Elaboración propia.

La identidad visual es la que ya uso con mis clientes en la versión web. El rojo de la marca va en la acción principal de cada pantalla, el blanco y los grises en todo lo demás y la fuente Anton solo en los títulos. Cada pantalla tiene una sola tarea, para que un cliente que nunca la ha visto sepa qué hacer sin leer instrucciones, y los botones principales tienen un tamaño táctil mínimo de 48 dp. La app final respeta los mockups casi por completo. El calendario de «Pedir cita» enseña solo las dos

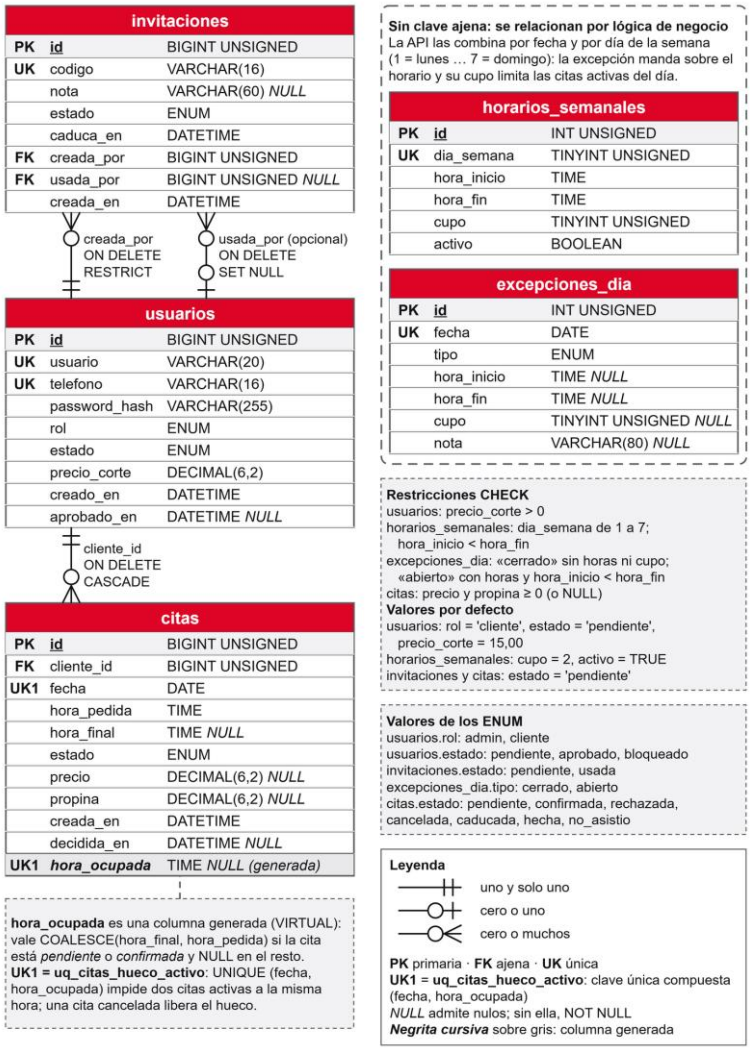
semanas reservables, sin flechas de mes, y la agenda del barbero se quedó en vista semanal.

5.2 Modelo de datos

La base de datos tiene cinco tablas relacionadas (figura 2). Clientes y barbero comparten `usuarios` y se distinguen por el rol, porque solo hay un barbero. `invitaciones` guarda los códigos de alta de un solo uso, `citas` cada cita con su hora pedida, su hora final, su estado y su precio, `horarios_semanales` el tramo y el cupo de cada día de la semana y `excepciones_dia` las vacaciones y los días sueltos de apertura. La tabla III.12 del anexo III detalla las restricciones de cada una.

Figura 2

Diagrama entidad-relación de la base de datos `clandestino`



Nota. Elaboración propia.

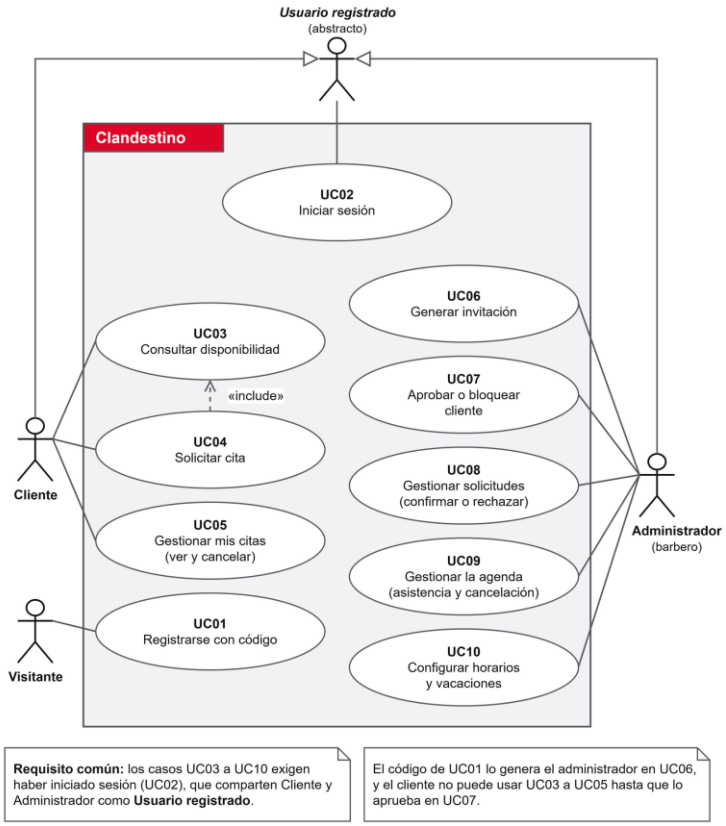
Quería que no pudiera haber dos citas activas a la misma hora, pero sí varias canceladas o rechazadas en esa hora. La aplicación original usa PostgreSQL, que lo resuelve con un índice único parcial, y MySQL no tiene ese tipo de índice. Lo resolví con una columna generada, `hora_ocupada`, que vale la hora de la cita mientras está pendiente o confirmada y pasa a `NULL` cuando deja de estarlo. Como MySQL admite varios `NULL` en un índice único, el índice `uq_citas_hueco_activo (fecha, hora_ocupada)` solo vigila las citas activas (Oracle, 2026). El precio se copia en la cita al marcar la asistencia, para que un cambio de precio posterior no altere las citas ya cobradas.

5.3 Casos de uso

Hay tres actores (figura 3). El visitante es una persona con invitación que aún no tiene cuenta, y cliente y administrador heredan de «usuario registrado», que es quien inicia sesión. Pedir cita incluye siempre consultar antes la disponibilidad.

Figura 3

Diagrama de casos de uso



Nota. Elaboración propia.

Tabla 2

Resumen de los casos de uso

Caso	Actor	Qué hace y cómo termina
UC01 Registrarse con código	Visitante	Crea su cuenta con un código vigente y queda pendiente de aprobación
UC02 Iniciar sesión	Usuario registrado	Recibe un token que la app guarda cifrado y entra a su panel o a la espera
UC03 Consultar disponibilidad	Cliente	Ve los días de las dos próximas semanas y las horas libres de cada uno
UC04 Solicitar cita	Cliente	La API comprueba horario, cupo y hueco y guarda la cita pendiente
UC05 Gestionar mis citas	Cliente	Ve sus citas y cancela una activa, que libera su hueco
UC06 Generar invitación	Administrador	Obtiene un código de 12 caracteres, válido 7 días y de un solo uso
UC07 Aprobar o bloquear cliente	Administrador	Cambia el estado del cliente, que rige desde su siguiente petición
UC08 Gestionar solicitudes	Administrador	Confirma con la hora pedida u otra, o rechaza
UC09 Gestionar la agenda	Administrador	Marca si el cliente vino, cancela citas futuras y crea citas a mano
UC10 Configurar horarios	Administrador	Edita tramo y cupo de cada día, vacaciones y días extra

Nota. Elaboración propia. La descripción completa, con flujos alternativos y errores de la API, está en el anexo III.

5.4 Clases y métodos principales

La API se organiza en paquetes por capa y la app en pantallas, modelos de vista y repositorios. La tabla 3 recoge las clases que llevan la lógica, y la estructura de carpetas está en el apartado 7.6.

Tabla 3

Clases y métodos principales

Clase	Métodos clave	Qué hace
`Enrutador` (API)	`registrar`, `despachar`	Busca la ruta de cada petición, comprueba rol y estado y convierte los errores en JSON
`FiltroAutenticacion` (API)	`autenticar`	Valida el token de la cabecera y carga al usuario
`TokenJwt` y `Contrasenas` (API)	`emitir`, `validar`, `hashear`, `verificar`	Firman los tokens con HMAC y protegen las contraseñas con PBKDF2

Clase	Métodos clave	Qué hace
`ServicioCitas` (API)	`pedir`, `confirmar`, `rechazar`, `marcarAsistencia`, `crearManual`	Reglas de la cita, con el cerrojo por fecha que protege el cupo
`ServicioHorarios` (API)	`diasDelRango`, `huecos`, `guardarHorarios`	Calcula qué días y horas quedan libres según horario, excepciones y citas
`RepositorioCitasJdbc` (API)	consultas con `PreparedStatement`	Lee y escribe las citas en MySQL dentro de la transacción del servicio
`ClienteApi` (app)	`enviar`	Hace la petición HTTP en un hilo aparte y devuelve el resultado al hilo principal
`ModeloVistaPedirCita` (app)	carga de días y huecos, envío de la solicitud	Guarda el estado de la pantalla en `LiveData` y sobrevive al giro

Nota. Elaboración propia.

6. METODOLOGÍA Y PLANIFICACIÓN

6.1 Ciclo de vida

El proyecto sigue un ciclo de vida incremental. Primero diseñé todo el sistema (datos, casos de uso, pantallas y contrato de la API) y después lo construí por piezas que se podían probar solas: la base de datos con sus pruebas, la API con las suyas y, por último, la app contra la API real. Cada incremento terminaba con algo que funcionaba y que el siguiente usaba sin cambios.

Para organizarme tomé algunas prácticas de Scrum (Schwaber y Sutherland, 2020) adaptadas a una sola persona. Sin equipo ni reuniones diarias, me quedé con la lista de trabajo ordenada por prioridad, los ciclos cortos con un objetivo concreto y la revisión al final de cada uno, porque los requisitos se fueron afinando sobre la marcha. Al dibujar los mockups o al escribir las pruebas aparecían detalles que el plan no cerraba, como qué hacer con una hora que ya ha pasado o cuánto dura un token, y un modelo en cascada estricto no me habría dejado volver atrás.

La lista tenía tres prioridades. La **P1** era el prototipo mínimo con el ciclo completo de la cita, la **P2** las mejoras si sobraba tiempo (cita creada por el barbero, cambio de @usuario, borrado de clientes, copia de seguridad y caducidad de solicitudes) y la **P3** lo opcional (resumen mensual de ingresos y caché local). La P1 está terminada y, de la P2, la cita que crea el barbero.

6.2 Requisitos de partida

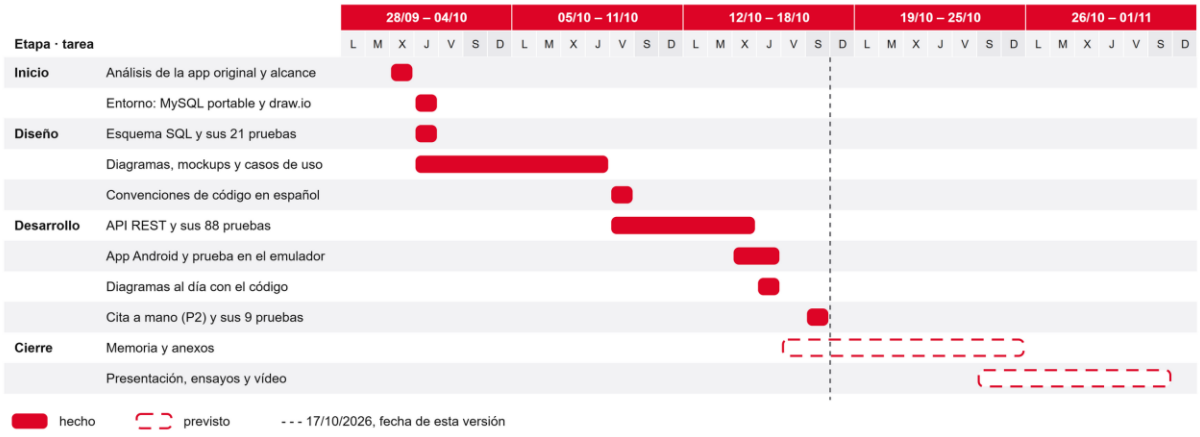
Para empezar necesitaba Android Studio con su emulador, el JDK 21, MySQL, draw.io y Git, además de lo aprendido en el ciclo sobre Java, Android, hilos y bases de datos. Los requisitos del producto salieron de mi propia forma de trabajar y de la aplicación web que ya uso. Revisé su modelo de datos, sus pantallas y sus reglas y decidí qué conservaba y qué simplificaba. Ahora todos entran con @usuario y contraseña, clientes y barbero comparten una tabla, el precio se guarda en la cita y el horizonte de reserva es fijo, de dos semanas. Las 14 decisiones de detalle que el plan no resolvía, como la duración del token o el límite de intentos, quedaron anotadas en un documento de reglas de negocio.

6.3 Planificación y tareas

Repartí el trabajo en las cuatro etapas que pide la guía (figura 4 y tabla 4). En el diagrama, las barras macizas son el trabajo hecho y las discontinuas lo que queda hasta la entrega.

Figura 4

Diagrama de Gantt del proyecto



Nota. Elaboración propia.

Tabla 4

Estimación de dedicación por etapas

Etapa	Tareas	Horas
Inicio	Estudio de la aplicación original, alcance e instalación del entorno	6

Etapa	Tareas	Horas
Diseño	Esquema SQL y sus pruebas, entidad-relación, casos de uso, requisitos, mockups, diagramas y contrato de la API	14
Desarrollo	API con sus pruebas, app de cliente y de barbero, recorrido en el emulador y cita a mano	40
Cierre	Memoria, anexos, presentación y vídeo de la defensa	18
Total		**78**

Nota. Elaboración propia.

El proyecto se ha compaginado con el trabajo. La dedicación aproximada ha sido de 3 horas al día de lunes a viernes y de entre 2 y 4 horas en total durante cada fin de semana, repartidas según la disponibilidad. Esto supone entre 17 y 19 horas semanales. Para un periodo de unos 30 días, tomando como referencia 22 días laborables y cuatro fines de semana, el esfuerzo se sitúa entre 74 y 82 horas. La tabla utiliza 78 horas como estimación central (66 horas entre semana y 12 durante los fines de semana).

El reparto por etapas es aproximado y no procede de un registro horario. Incluye consultas de documentación, pruebas y correcciones. Las 18 horas de cierre contemplan también el ensayo y la grabación pendientes, de modo que el total corresponde al esfuerzo previsto para terminar el proyecto. Las fechas del diario sitúan los hitos documentados; cada bloque puede reunir trabajo de varias tardes y las fases pueden solaparse.

6.4 Control de versiones y copias de seguridad

El proyecto está en el repositorio público https://github.com/Jhonnatanospina/clandestino-tfg, en la rama `main`. El primer commit agrupa la versión inicial del TFG. Los dos siguientes añaden la dirección del repositorio a la documentación y el vídeo de respaldo de la demostración. El historial permite recuperar esas versiones, aunque concentrar tanto trabajo en el primer commit dificulta seguir los cambios pequeños. Las modificaciones posteriores deben guardarse en commits separados por tarea.

El repositorio no incluye ningún secreto, porque las claves están en un fichero `.env` excluido con `.gitignore` y solo se sube un `.env.example` con los nombres de las variables. La base de datos se reconstruye en cualquier momento desde `database/schema.sql` y `database/seed.sql` con `scripts\bd.cmd`, que pide

confirmación antes de borrar nada, y los diagramas y mockups salen de scripts de Python que se pueden volver a ejecutar.

No consta una periodicidad de copias externas de la carpeta ni de los datos de uso. Los scripts recuperan la estructura y los datos iniciales, pero no las citas y los clientes añadidos después. Antes de utilizar la aplicación con datos reales habrá que guardar exportaciones periódicas con `mysqldump` fuera de la carpeta de trabajo y comprobar su restauración. Su automatización queda para la segunda fase.

7. DESARROLLO

7.1 Tecnologías y versiones

Tabla 5

Tecnologías y versiones utilizadas

Herramienta	Versión	Para qué y por qué
Java	21 en la API y 17 en la app	Lenguaje principal del ciclo y uno de los dos nativos de Android
Android Studio	Panda 4, con su JBR 21.0.10	Entorno oficial de Android, con emulador incluido
Gradle y Android Gradle Plugin	9.4.1 y 9.2.0	Compilan la app y la API con las mismas herramientas
SDK de Android	compileSdk 36.1, targetSdk 36, minSdk 26	La app funciona desde Android 8.0
AndroidX y Material	appcompat 1.8.0, material 1.14.0, lifecycle 2.11.0	Componentes de Material 3, listas y MVVM
MySQL y Connector/J	9.7.2 LTS portable y 9.7.0	Base relacional del módulo de Bases de datos, accedida por JDBC
org.json y JUnit	20260814; JUnit 6.1.3 (API) y 4.13.2 (app)	JSON sin dependencias pesadas y pruebas automáticas
draw.io	31.5.3	Diagramas UML y mockups

Nota. Elaboración propia.

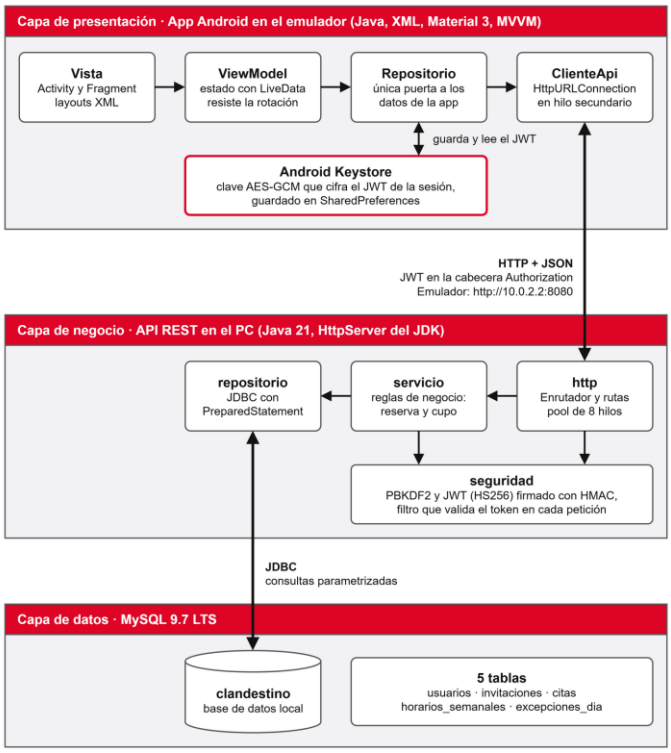
La API no usa ningún framework web. El servidor es el `HttpServer` que trae el JDK (Oracle, s. f.-a) y los datos se leen con JDBC y `PreparedStatement` (Oracle, s. f.-b). Lo elegí así para programar a mano lo que un framework daría hecho (rutas, filtros, JSON y transacciones) y poder explicarlo en la defensa. Por lo mismo, la app usa `HttpURLConnection` en lugar de Retrofit.

7.2 Arquitectura y diagrama de clases

El sistema tiene tres capas (figura 5). La app solo conoce la API, y la API entra en la base de datos con un usuario de MySQL que puede leer y modificar filas pero no crear ni borrar tablas.

Figura 5

Arquitectura del sistema en tres capas



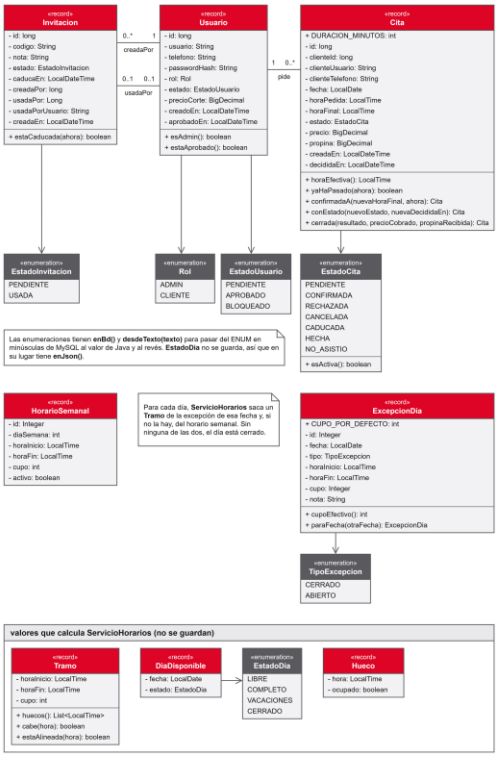
Nota. Elaboración propia.

En la API, una petición entra por el `Enrutador`, que pasa la cabecera al filtro de autenticación y con el usuario ya cargado comprueba el rol y el estado. Después la ruta llama al servicio y este al repositorio. Si una regla no se cumple, el servicio lanza un `ErrorNegocio` con su código HTTP y el `Enrutador` lo convierte en `{"error", "mensaje"}`, así que la app recibe siempre el mismo formato. La app sigue el patrón MVVM que recomienda Android (Android Developers, 2026c). Cada pantalla observa un `ViewModel` que guarda el estado en `LiveData`, y este pide los datos a un repositorio que usa `ClienteApi` en un hilo secundario, porque Android no permite usar la red en el hilo de la interfaz (Android Developers, 2026b).

La figura 6 muestra el modelo de clases de la API. Los diagramas completos de la API y de la app están en el anexo III (figuras III.1 a III.6), y los rehíce al final a partir de las clases compiladas para que tengan los mismos nombres que el código.

Figura 6

Diagrama de clases del modelo de la API



Nota. Elaboración propia.

7.3 Requisitos funcionales y no funcionales

Escribí los requisitos en la fase de diseño, cada uno con su identificador. Hay **30 funcionales** de la primera fase (RF-01 a RF-30), todos implementados, y el anexo III relaciona cada caso de uso con su pantalla, sus rutas y sus requisitos. De los 13 de segunda fase está hecha la cita a mano (RF-P2-02). Los **53 no funcionales** se reparten en seguridad (13), rendimiento (8), usabilidad (11), compatibilidad (7), mantenibilidad (8) y privacidad (6), y los que no he cumplido se indican en el apartado 7.7.

Los de seguridad siguen OWASP MASVS 2.1.0 (OWASP Mobile Application Security, 2024) y la guía de OWASP sobre contraseñas (OWASP Cheat Sheet Series, s. f.), y la tabla III.13 del anexo III los detalla por categoría. Las contraseñas se guardan con PBKDF2-HMAC-SHA256 y 600.000 iteraciones, y el token de sesión es un JWT

que programé a mano con la clase `Mac` siguiendo la RFC 7519 (Jones et al., 2015) y la RFC 8725 (Sheffer et al., 2020), de modo que la API solo acepta HS256. La app guarda el token cifrado con una clave del Android Keystore (Android Developers, 2026a), y tras 5 intentos fallidos en 15 minutos la API responde 429. Cada cálculo de PBKDF2 tarda unos 0,75 s en mi ordenador, así que el servidor atiende con 8 hilos para que un inicio de sesión no deje en espera a las demás peticiones.

7.4 Concurrencia: el cupo diario

El caso más delicado aparece cuando dos clientes piden cita para el mismo día a la vez. El índice único impide dos citas activas a la misma hora (apartado 5.2), pero ningún índice cuenta filas, así que no puede impedir que se supere el cupo. Si dos peticiones cuentan las citas del día al mismo tiempo, las dos ven sitio y las dos insertan.

`ServicioCitas` lo resuelve con un bloque `synchronized` sobre un cerrojo por fecha guardado en un `ConcurrentHashMap`. Dentro van, en una sola transacción, la cuenta de citas activas, la inserción y el `COMMIT`. Las peticiones del mismo día se ponen en fila, las de días distintos no se esperan, y si aun así llegaran dos citas a la misma hora, MySQL las frena con el error 1062, que la API traduce a `409 hueco_ocupado`. La prueba `PruebaConcurrenciaCupo` lanza 5 hilos a la vez con un `CountDownLatch` y un repositorio «lento» que espera 300 ms después de contar, para que la carrera ocurra siempre (tabla 6). El diagrama de secuencia de esta parte es la figura III.11 del anexo III.

Tabla 6

Resultados de la prueba de concurrencia del cupo

Caso	Resultado
Cupo 1, horas distintas, sin `synchronized`	Se crean 5 citas en un día con cupo 1
Cupo 1, horas distintas, con `synchronized`	Se crea 1 cita y las otras 4 reciben `dia_lleno`
Cupo 5, misma hora, sin `synchronized`	Se crea 1 cita y las otras 4 reciben `hueco_ocupado` gracias al índice único

Nota. Elaboración propia.

El cerrojo vive en la memoria del proceso. Con dos copias de la API habría que llevar el bloqueo a la base de datos, por ejemplo con `SELECT ... FOR UPDATE` sobre una fila por fecha.

7.5 Diario de desarrollo

El diario completo está en el anexo V. Estas son las entradas con los problemas que más me enseñaron.

30-09 y 01-10 · Inicio y diseño. Cerré el alcance y dejé fuera métricas, nóminas y avisos. MySQL no tiene índices únicos parciales y lo resolví con la columna `hora_ocupada`. Una revisión de coherencia entre diagramas, mockups y requisitos encontró 12 fallos, como un mockup que enseñaba el precio al cliente, y los corregí antes de programar.

09-10 a 14-10 · API. Decidí escribir todo el código en español. `HttpServer` atiende en un solo hilo si no se le da un `Executor` y solo reparte por prefijo, lo que me obligó a escribir un `Enrutador` propio para rutas como `/api/citas/{id}/cancelar`. Sin `ResolverStyle.STRICT`, el 30 de febrero se convertía en el 2 de marzo. La API quedó con 88 pruebas en verde.

15-10 · App en el emulador. Recorrí el flujo completo con datos reales. Las horas de «Pedir cita» salían pegadas a la izquierda, y un registro de depuración mostró que Android ignoraba `textAlignment` porque el manifiesto tenía `supportsRtl="false"`.

17-10 · Cita a mano. Hice la primera función de la segunda fase, la cita que el barbero mete para quien llama por teléfono, con 9 pruebas nuevas. Lo más delicado fue el giro de pantalla, porque el diálogo tenía que conservar lo elegido sin que el desplegable filtrara la lista de clientes.

7.6 Organización de carpetas

```
clandestino-tfg/
├─ api/        API REST (Gradle): http, seguridad, servicio, repositorio y modelo
├─ android/    App Android (Gradle): ui, modelovista, datos, modelo y utilidades
├─ database/   schema.sql, seed.sql y pruebas del esquema
├─ scripts/    Arranque de MySQL, carga de la base y usuario de la app
└─ docs/       Diseño, diagramas, mockups, capturas y memoria
```

Las claves están en un `.env` que no se sube al repositorio, y el `.env.example` recoge los nombres de las variables sin valores.

7.7 Pruebas

Tabla 7

Resumen de las pruebas

Conjunto	Qué comprueba	Resultado
Esquema (21 pruebas SQL)	Índice único de citas activas, restricciones `CHECK` y claves foráneas	21 de 21 correctas
API, unitarias e integración contra MySQL	Contraseñas, token, limitador, huecos, horarios, servicio de citas (32 pruebas), usuarios, concurrencia y la API completa por HTTP	97 de 97 en verde
App, unitarias	Modelos desde JSON, calendario, fechas, validaciones y horas del desplegable	26 de 26 en verde
Recorrido manual en el emulador	Flujo completo de cliente y barbero, cita a mano, tema oscuro y cierre de sesión	Correcto, con 17 capturas

Nota. Elaboración propia.

Las pruebas de integración usan una base de datos aparte y un reloj que se puede mover, para comprobar reglas como «no se puede pedir una hora pasada» sin depender de la hora real. Los casos manuales, con sus pasos y su resultado, están en el anexo IV. Lint no da errores. No llegué a hacer pruebas instrumentadas con Espresso ni a medir la consulta de huecos con `EXPLAIN`, la app no tiene diseño específico para horizontal o tableta y la comunicación no va cifrada con HTTPS. Las recojo como mejoras en las conclusiones.

8. CONCLUSIONES Y PROYECCIÓN A FUTURO

8.1 Cumplimiento de los objetivos

Los cinco objetivos específicos se han cumplido en lo esencial.

Tabla 8

Grado de cumplimiento de los objetivos

Objetivo	Grado de cumplimiento
1. Base de datos que impida la doble reserva	Cumplido. La columna generada `hora_ocupada` con su índice único bloquea dos citas activas a la misma hora, y las 21 pruebas del esquema lo confirman.
2. API REST en Java sin frameworks, con autenticación y roles	Cumplido. Tiene 24 rutas, contraseñas con PBKDF2, token firmado con HMAC y comprobación de rol y estado en cada petición.
3. Cupo diario seguro ante peticiones simultáneas	Cumplido. La prueba con 5 hilos crea 5 citas en un día con cupo 1 si se quita el `synchronized` y solo 1 si se mantiene.

Objetivo	Grado de cumplimiento
4. App Android con MVVM que no bloquee el hilo principal	Cumplido. Tiene todas las pantallas de cliente y barbero, más el diálogo de la cita a mano, y toda la red va en hilos secundarios.
5. Comprobación con pruebas y en el emulador	Cumplido. Las 97 pruebas de la API y las 26 de la app pasan, y el recorrido completo está documentado con capturas.

Nota. Elaboración propia.

8.2 Limitaciones

El prototipo funciona solo en local. Antes de usarlo con mis clientes y de ofrecerlo a otros barberos habría que resolver varias cosas:

- **No hay HTTPS.** La app habla con la API por HTTP sin cifrar, algo aceptable en el emulador pero no en una red real. Habría que poner la API detrás de un certificado TLS.
- **El cerrojo del cupo vive en la memoria de la API.** Con una sola copia del servidor funciona, pero con dos habría que pasar el bloqueo a la base de datos.
- **Faltan comprobaciones de calidad previstas en el plan.** No hay pruebas instrumentadas de la interfaz, no he medido el rendimiento con `EXPLAIN` ni con `StrictMode`, y la app no tiene animaciones ni un diseño pensado para horizontal o tableta.
- **Las solicitudes pendientes no caducan solas.** Si el barbero no responde, la cita sigue ocupando su hueco hasta que la rechaza.
- **Dos citas pueden solaparse media hora.** El índice único solo impide la misma hora exacta, así que al confirmar a otra hora o al meter una cita a mano puede quedar una a las 10:00 y otra a las 10:30. Es la regla del original (DP-04), y el barbero lo ve en la agenda antes de decidir.
- **No hay copia de seguridad automática** de la base de datos.

8.3 Trabajo futuro

Las líneas de mejora siguen el orden de prioridad del proyecto. De la segunda fase ya está hecha la cita que el barbero crea para los clientes que llaman por teléfono, así que lo siguiente serían las demás funciones P2 especificadas (cambio de @usuario, borrado de clientes, caducidad automática de solicitudes y copia de seguridad con `mysqldump`) y, por último, las que dejé fuera del alcance al principio:

- Avisos al cliente cuando el barbero confirma o rechaza su cita.
- El resumen mensual de ingresos y citas atendidas, que ya tiene los datos necesarios porque el precio se guarda en cada cita.
- Publicar la API en un servidor con HTTPS.
- Que otro barbero pueda montar su propia copia, con su nombre y sus colores, sin tocar el código.

8.4 Valoración personal

[completar por el alumno: qué ha aprendido, qué le ha costado más, qué haría distinto si empezara de nuevo y cómo encaja el proyecto con su trabajo como barbero en casa.]

9. BIBLIOGRAFÍA

AgendaPro. (s. f.). *Planes*. Recuperado el 16 de octubre de 2026 de https://agendapro.com/es-es/planes

Android Developers. (2026a, 6 de marzo). *Android Keystore system*. https://developer.android.com/privacy-and-security/keystore

Android Developers. (2026b, 26 de febrero). *Asynchronous work with Java threads*. https://developer.android.com/guide/background/threading

Android Developers. (2026c, 5 de octubre). *Guide to app architecture*. https://developer.android.com/topic/architecture

Android Developers. (2026d, 28 de agosto). *Network security configuration*. https://developer.android.com/privacy-and-security/security-config

Booksy. (s. f.). *Precios*. Recuperado el 16 de octubre de 2026 de https://biz.booksy.com/es-es/precios

Calendly. (s. f.). *Pricing*. Recuperado el 16 de octubre de 2026 de https://calendly.com/pricing

Comisión Nacional de los Mercados y la Competencia. (2026, 22 de mayo). *Instagram y TikTok, las redes sociales que más crecieron en España en 2025* [Nota de prensa]. https://cnmc.es/prensa/panel-hogares-ott-internet-20260522

Fresha. (s. f.). *Pricing*. Recuperado el 16 de octubre de 2026 de https://www.fresha.com/pricing

Instituto Nacional de Estadística. (s. f.). *Empresas por condición jurídica, actividad principal (grupos CNAE 2009) y estrato de asalariados* (Tabla 73019) [Conjunto de datos]. Directorio Central de Empresas (DIRCE). Recuperado el 16 de octubre de 2026 de https://www.ine.es/jaxiT3/Tabla.htm?t=73019

Instituto Nacional de Estadística. (2025, 20 de noviembre). *Encuesta sobre Equipamiento y Uso de Tecnologías de la Información y Comunicación en los Hogares. Año 2025* [Nota de prensa]. https://www.ine.es/dyngs/Prensa/TICH2025.htm

Jones, M., Bradley, J. y Sakimura, N. (2015). *JSON Web Token (JWT)* (RFC 7519). Internet Engineering Task Force. https://doi.org/10.17487/RFC7519

Oracle. (s. f.-a). *HttpServer (Java SE 21 & JDK 21)*. Recuperado el 16 de octubre de 2026 de https://docs.oracle.com/en/java/javase/21/docs/api/jdk.httpserver/com/sun/net/httpserver/HttpServer.html

Oracle. (s. f.-b). *PreparedStatement (Java SE 21 & JDK 21)*. Recuperado el 16 de octubre de 2026 de https://docs.oracle.com/en/java/javase/21/docs/api/java.sql/java/sql/PreparedStatement.html

Oracle. (2026). CREATE TABLE and generated columns (sección 15.1.25.8). En *MySQL 9.7 reference manual*. Recuperado el 16 de octubre de 2026 de https://dev.mysql.com/doc/refman/9.7/en/create-table-generated-columns.html

OWASP Cheat Sheet Series. (s. f.). *Password storage cheat sheet*. Recuperado el 16 de octubre de 2026 de https://cheatsheetseries.owasp.org/cheatsheets/Password_Storage_Cheat_Sheet.html

OWASP Mobile Application Security. (2024). *OWASP Mobile Application Security Verification Standard (MASVS)* (versión 2.1.0). https://mas.owasp.org/MASVS/

Real Decreto 450/2010, de 16 de abril, por el que se establece el título de Técnico Superior en Desarrollo de Aplicaciones Multiplataforma y se fijan sus enseñanzas mínimas. *Boletín Oficial del Estado*, núm. 123, de 20 de mayo de 2010, pp. 43776-43846. https://www.boe.es/eli/es/rd/2010/04/16/450

Reglamento (UE) 2016/679 del Parlamento Europeo y del Consejo, de 27 de abril de 2016, relativo a la protección de las personas físicas en lo que respecta al tratamiento de datos personales y a la libre circulación de estos datos y por el que se deroga la Directiva 95/46/CE (Reglamento general de protección de datos). *Diario Oficial de la Unión Europea*, L 119, de 4 de mayo de 2016, pp. 1-88. https://eur-lex.europa.eu/eli/reg/2016/679/oj

Schwaber, K. y Sutherland, J. (2020). *La Guía Scrum: La guía definitiva de Scrum: Las reglas del juego* (D. Martin y D. Tardini, Trads.). https://scrumguides.org/docs/scrumguide/v2020/2020-Scrum-Guide-Spanish-European.pdf

Setmore. (s. f.). *Pricing for appointment scheduling software*. Recuperado el 16 de octubre de 2026 de https://www.setmore.com/pricing

Sheffer, Y., Hardt, D. y Jones, M. (2020). *JSON Web Token best current practices* (RFC 8725). Internet Engineering Task Force. https://doi.org/10.17487/RFC8725

Square. (s. f.). *Citas Square: Precios*. Recuperado el 16 de octubre de 2026 de https://squareup.com/es/es/appointments/pricing

Treatwell. (s. f.). *Precios Treatwell Connect: paga 0% por clientes fieles*. Recuperado el 16 de octubre de 2026 de https://www.treatwell.es/partners/precios/

[completar: si el título que cursa el alumno es el actualizado, sustituir el Real Decreto 450/2010 por el Real Decreto 405/2023, de 29 de mayo (*Boletín Oficial del Estado*,

núm. 132, de 3 de junio de 2023, pp. 79000-79096, https://www.boe.es/eli/es/rd/2023/05/29/405), y cambiar la cita del apartado 3.1. Borrar esta nota antes de entregar.]

10. ANEXOS

Estos son los cinco anexos de la memoria. Los dos primeros, el manual de instalación y el manual de usuario, son los que exige la guía. Los otros tres amplían el diseño técnico, recogen las pruebas hechas a mano y guardan el diario de desarrollo. El código fuente completo está en el repositorio público https://github.com/Jhonnatanospina/clandestino-tfg.

Anexo I. Manual de instalación

Este anexo explica cómo poner en marcha el prototipo en un ordenador con Windows: la base de datos, la API y la app en el emulador de Android Studio. Los pasos se han comprobado en el equipo de desarrollo. Al final de cada bloque hay una comprobación para saber si ha ido bien antes de pasar al siguiente.

I.1 Requisitos

Tabla I.1

Requisitos de instalación

Elemento	Requisito
Sistema operativo	Windows 10 u 11 de 64 bits
Memoria	8 GB de RAM como mínimo (el emulador reserva unos 2 GB)
Android Studio	Panda 4 o posterior, que admita Android Gradle Plugin 9.2. Trae su propio JDK 21 (JBR), que también sirve para la API
SDK de Android	Plataforma 36.1 y una imagen de sistema de Android 16 (API 36) para el emulador
MySQL	MySQL Community Server 9.7.2 LTS, en ZIP para Windows de 64 bits
Puertos libres	3306 (MySQL) y 8080 (API), los dos solo en `127.0.0.1`

Nota. Elaboración propia.

I.2 Obtener el proyecto

El código está en el repositorio público https://github.com/Jhonnatanospina/clandestino-tfg.

```
git clone https://github.com/Jhonnatanospina/clandestino-tfg.git
C:\Dev\clandestino-tfg
```

Conviene usar una ruta corta y sin espacios ni tildes, porque Gradle y algunos scripts de MySQL dan problemas con ellas. En el resto del anexo se supone que el proyecto está en `C:\Dev\clandestino-tfg`.

I.3 Base de datos

1. Descomprimir el ZIP de MySQL en `C:\Dev\mysql-9.7.2-winx64`.
2. Crear la carpeta `C:\Dev\mysql-data` y dentro un fichero `my.ini` con este contenido:

```
[mysqld]
basedir=C:/Dev/mysql-9.7.2-winx64
datadir=C:/Dev/mysql-data/datos
port=3306
bind-address=127.0.0.1
mysqlx=OFF
character-set-server=utf8mb4
collation-server=utf8mb4_0900_ai_ci
innodb_buffer_pool_size=128M
max_connections=30
performance_schema=OFF
log-error=C:/Dev/mysql-data/mysql-error.log

[client]
port=3306
default-character-set=utf8mb4
```

   `bind-address=127.0.0.1` hace que MySQL solo acepte conexiones del propio ordenador.

3. Inicializar la carpeta de datos. El comando escribe en `mysql-error.log` una contraseña temporal para `root`:

```
C:\Dev\mysql-9.7.2-winx64\bin\mysqld.exe --defaults-file=C:\Dev\mysql-
data\my.ini --initialize
```

4. Arrancar MySQL con el script del proyecto:

```
C:\Dev\clandestino-tfg\scripts\mysql.cmd iniciar
```

5. Entrar con la contraseña temporal y cambiarla por una propia:

```
C:\Dev\mysql-9.7.2-winx64\bin\mysql.exe -u root -p
ALTER USER 'root'@'localhost' IDENTIFIED BY 'nueva_contraseña';
```

Si MySQL está en otra carpeta, basta con definir las variables de entorno `MYSQL_BASE` y `MYSQL_DATOS` antes de usar los scripts.

Comprobación: `scripts\mysql.cmd estado` responde «MySQL está en marcha.».

I.4 Fichero de configuración `.env`

1. Copiar `.env.example` como `.env` en la raíz del proyecto. Este fichero no se sube nunca al repositorio.
2. Rellenar los valores:

- `DB_HOST=127.0.0.1`, `DB_PORT=3306` y `DB_NAME=clandestino`.
- `DB_USER` y `DB_PASSWORD`: el usuario con el que entrará la API (por ejemplo `clandestino_app`) y una contraseña inventada. El usuario se crea en el paso siguiente.
- `DB_ROOT_PASSWORD`: la contraseña de `root` del paso I.3.
- `JWT_SECRET`: una clave aleatoria de al menos 32 caracteres. Se puede generar en PowerShell con:

```
$b = New-Object byte[] 48;
[Security.Cryptography.RandomNumberGenerator]::Create().GetBytes($b);
[Convert]::ToBase64String($b)
```

- `API_PORT=8080`.

I.5 Crear las tablas y el usuario de la API

```
cd C:\Dev\clandestino-tfg
scripts\bd.cmd cargar -Confirmo
scripts\bd.cmd usuario-app
```

El primero crea la base `clandestino` con sus cinco tablas y carga un horario de ejemplo (de lunes a jueves, de 16:00 a 22:00, con cupo 2). Borra las tablas si ya existían, y por eso exige `-Confirmo`. El segundo crea el usuario de `DB_USER`, que solo puede consultar y modificar filas, nunca crear ni borrar tablas.

Comprobación (opcional): `scripts\bd.cmd probar` pasa las 21 pruebas del esquema en una base aparte, sin tocar la real.

I.6 API

1. Abrir PowerShell en la carpeta `api` y apuntar `JAVA_HOME` al JDK de Android Studio:

```
cd C:\Dev\clandestino-tfg\api
$env:JAVA_HOME = 'C:\Program Files\Android\Android Studio\jbr'
```

2. Crear la cuenta del barbero (solo la primera vez). Pide la contraseña por teclado:

```
.\gradlew.bat crearAdmin --args="barbero +34600111222"
```

3. Arrancar la API:

```
.\gradlew.bat run
```

Comprobación: en otra ventana, `curl http://127.0.0.1:8080/api/salud` devuelve `{"estado":"ok","base_de_datos":"ok"}`. Si MySQL está parado devuelve un error 503 `base_de_datos_no_disponible`.

Pruebas (opcional): `scripts\bd.cmd base-tests-api` crea la base vacía de pruebas, y después `.\gradlew.bat test` pasa las 97 pruebas de la API.

I.7 App en el emulador

1. Abrir la carpeta `C:\Dev\clandestino-tfg\android` en Android Studio y esperar a que termine la sincronización de Gradle.
2. En *Device Manager*, crear un dispositivo virtual con una imagen de Android 16 (API 36). Conviene ponerle la zona horaria Europe/Madrid, porque la API trabaja siempre en hora de Madrid y así la app y la API coinciden.
3. Con la API en marcha, pulsar *Run* con la configuración `app` en modo *debug*.

La versión de depuración llama a la API en `http://10.0.2.2:8080`, que es la dirección con la que el emulador ve al ordenador. Para probar en un móvil real por cable se usa `adb reverse tcp:8080 tcp:8080` y se cambia esa URL por `http://localhost:8080` en `android/app/build.gradle`.

También se puede instalar el APK de depuración sin abrir Android Studio. Se genera con `.\gradlew.bat assembleDebug` desde la carpeta `android` y queda en `android/app/build/outputs/apk/debug/app-debug.apk`.

Comprobación: la app abre la pantalla de inicio de sesión y entra con la cuenta del barbero creada en el paso I.6.

I.8 Problemas frecuentes

Tabla I.2

Problemas frecuentes de instalación

Síntoma	Causa y solución
La app dice que no hay conexión	La API no está arrancada o escucha en otro puerto. Comprobar `/api/salud` desde el PC
`/api/salud` devuelve 503	MySQL está parado: `scripts\mysql.cmd iniciar`
La API no arranca y dice «Falta … en el .env»	Falta el `.env` o una variable obligatoria (`DB_USER`, `DB_PASSWORD`, `JWT_SECRET`)
El selector de fechas sale en inglés	El emulador está en inglés. En un móvil en español no pasa

Nota. Elaboración propia.

Anexo II. Manual de usuario

Este manual explica cómo se usa Clandestino desde el móvil, primero como cliente y después como barbero. Las capturas son de la app funcionando en el emulador con datos de prueba.

II.1 Entrar en la aplicación

Iniciar sesión. Al abrir la app aparece la pantalla de entrada (figura II.1). Se escribe el @usuario y la contraseña y se pulsa «Entrar». Si la sesión ya estaba abierta, la app pasa directamente al panel. El icono del ojo muestra u oculta la contraseña.

Crear una cuenta. Solo se puede con un código de invitación que da el barbero. Desde la pantalla de entrada se pulsa «Crear cuenta con código» y se rellenan los cuatro campos (figura II.2):

- **Código de invitación**: 12 caracteres, válido durante 7 días y para una sola persona.
- **Usuario**: de 3 a 20 caracteres, solo minúsculas, números y guion bajo.
- **Teléfono**: con el prefijo del país, por ejemplo +34.
- **Contraseña**: de 8 caracteres como mínimo.

Cada campo avisa mientras se escribe si el formato no es correcto.

Esperar la aprobación. Después de registrarse, la app muestra una pantalla de espera (figura II.3) hasta que el barbero aprueba la cuenta. Con el botón «Comprobar de nuevo», o deslizando hacia abajo, la app pregunta otra vez si ya está aprobada. Si el barbero bloquea la cuenta, esta misma pantalla lo indica.

Figura II.1
Entrada

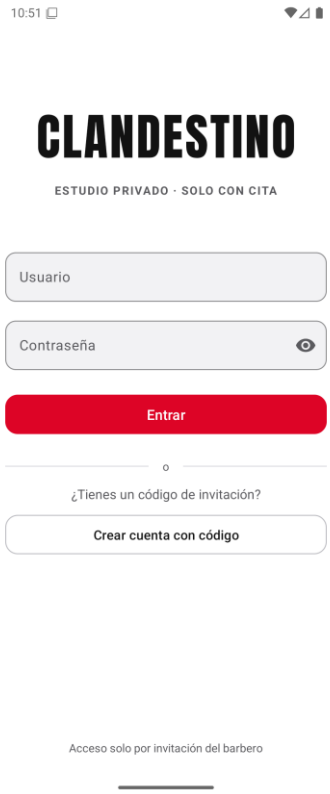
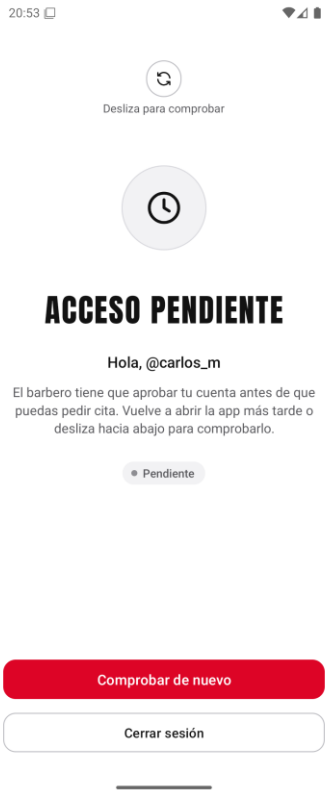


Figura II.2
Registro con código



Figura II.3
Acceso pendiente



Nota. Las figuras II.1 a II.3 son de elaboración propia.

II.2 Cliente

El panel del cliente tiene tres pestañas en la barra inferior: «Mis citas», «Pedir» y «Ajustes».

Mis citas (figura II.4). Muestra las próximas citas y, debajo, el historial. Cada tarjeta indica el día, la hora y el estado (pendiente, confirmada, hecha, cancelada...). Una cita pendiente o confirmada se puede anular con «Cancelar solicitud» o «Cancelar cita». La app pide confirmación antes de hacerlo y la hora queda libre para otro cliente. Para ver si el barbero ya ha confirmado una cita, se pulsa el botón de recargar o se desliza la lista hacia abajo.

Pedir cita (figura II.5). El calendario enseña las dos próximas semanas:

- Un día con borde está libre.
- Un día con fondo gris está completo.
- Un día con borde azul es de vacaciones.
- El borde rojo marca el día de hoy.
- Los días sin borde están cerrados y no se pueden elegir.

Al tocar un día libre aparecen sus horas. Se elige una y se pulsa el botón «Pedir cita», que repite el día y la hora para evitar errores. La cita queda pendiente hasta que el barbero la confirme. Si otra persona ha cogido esa hora un instante antes, la app avisa y vuelve a cargar las horas libres.

Ajustes (figura II.6). Muestra la cuenta y su estado y permite elegir entre tema claro y oscuro y cerrar la sesión.

Figura II.4
Mis citas

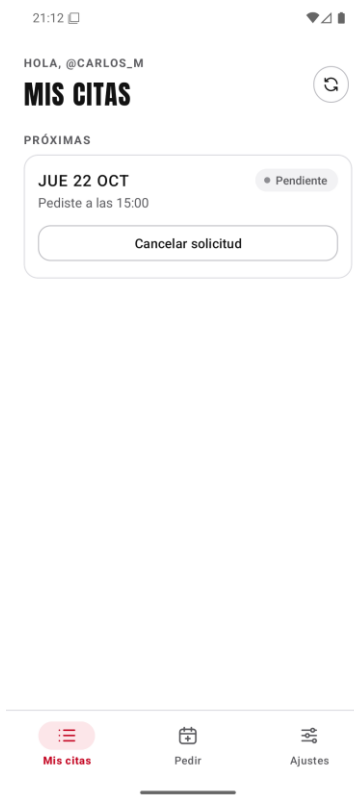


Figura II.5
Pedir cita

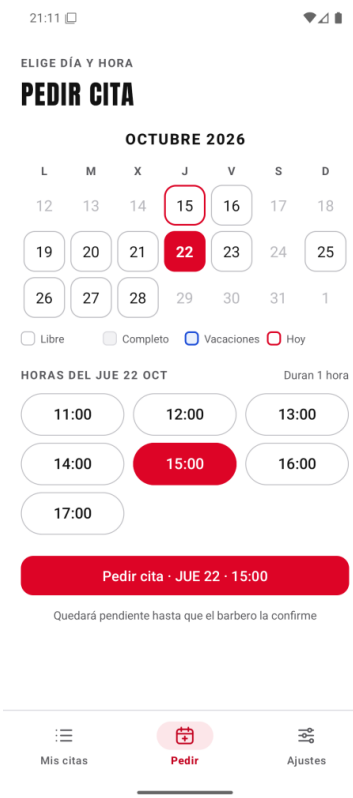
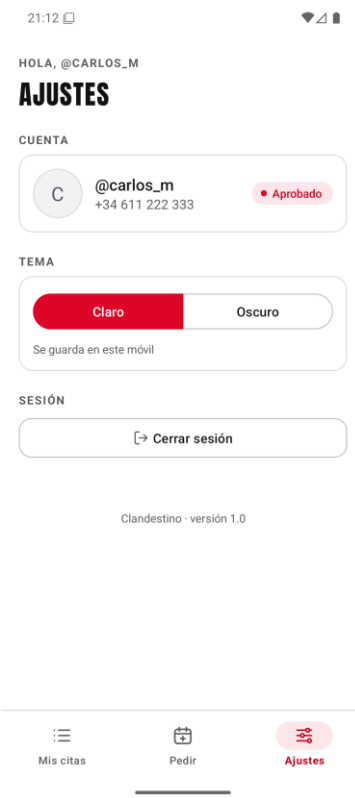


Figura II.6
Ajustes



Nota. Las figuras II.4 a II.6 son de elaboración propia.

II.3 Barbero

El panel del barbero tiene cuatro pestañas: «Solicitudes», «Agenda», «Horarios» y «Clientes». Para cerrar la sesión se usa el botón de salida que hay arriba en «Clientes».

Solicitudes (figura II.7). Lista las citas pendientes con el cliente, el día y la hora que pidió. En «Hora final» se puede dejar esa hora o elegir otra del mismo día. Después se pulsa «Confirmar» o «Rechazar». Si la hora elegida ya está ocupada por otra cita, la app lo avisa y no confirma.

Agenda (figura II.8). Muestra la semana con flechas para avanzar o retroceder. Los días que tienen citas llevan un punto rojo. Al tocar un día se ven sus citas confirmadas con la hora, el cliente y su teléfono. Cuando la hora de una cita ya ha pasado, se activan «Asistió» y «No vino». Una cita futura se puede cancelar. Arriba está el botón «Nueva cita», que se explica a continuación.

Figura II.7
Solicitudes

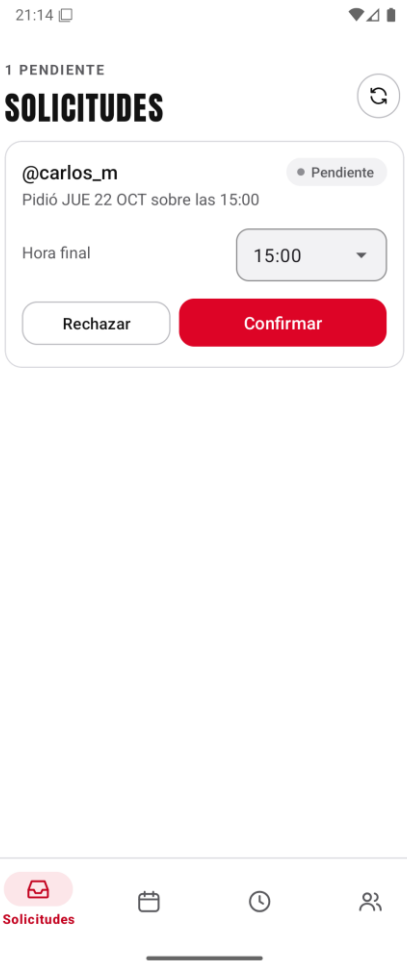
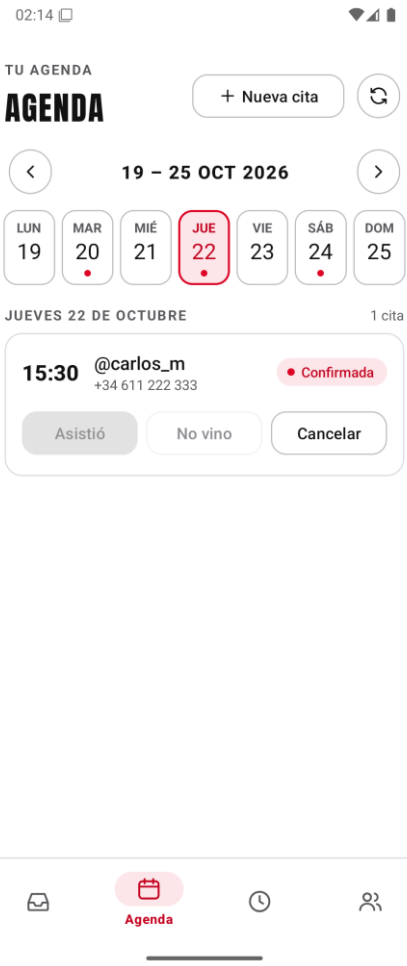


Figura II.8
Agenda



Nota. Las figuras II.7 y II.8 son de elaboración propia.

Nueva cita (figuras II.9 a II.11). Sirve para dar cita a un cliente que llama por teléfono o la pide en persona. En el diálogo se elige el cliente entre los aprobados, el día en un calendario que empieza en hoy y la hora en una lista que va de 08:00 a 22:00 cada media hora (figura II.9). Sale ya elegido el día que se estaba mirando en la agenda, y si aún no hay clientes aprobados, el diálogo lo indica. La cita se guarda confirmada aunque ese día esté completo o cerrado, y al cliente le aparece en «Mis citas». Al guardar, la agenda salta a ese día para que se vea la cita nueva (figura II.10). Si a esa hora ya hay otra cita, la app avisa con «Esa hora ya está cogida.» y no

guarda nada (figura II.11), así que basta con abrir otra vez «Nueva cita» y elegir otra hora.

Figura II.9
Nueva cita

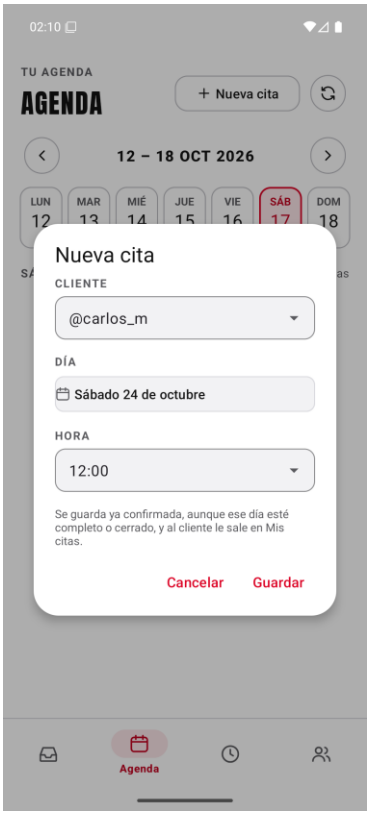


Figura II.10
Cita guardada en la agenda

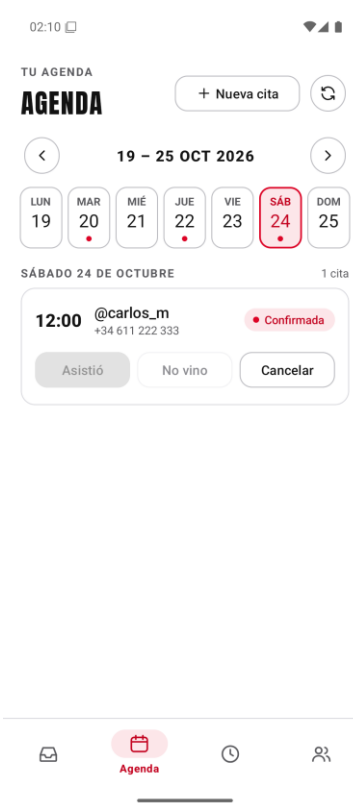
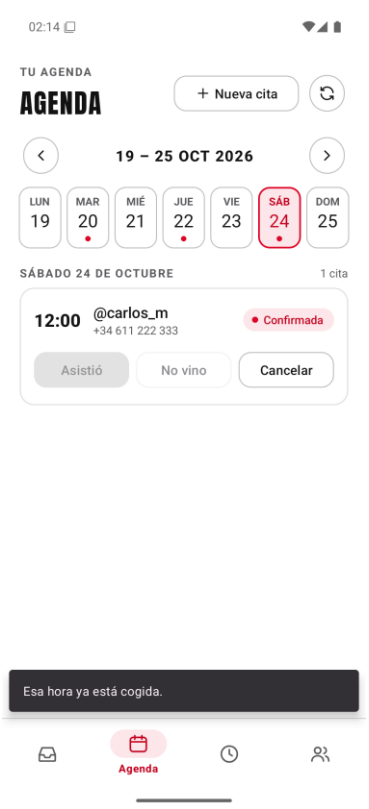


Figura II.11
Hora ya cogida



Nota. Las figuras II.9 a II.11 son de elaboración propia.

Horarios (figura II.12). En la plantilla semanal cada día tiene un interruptor para abrirlo o cerrarlo, su hora de inicio, su hora de fin y su cupo, que es el número máximo de citas de ese día. Al tocar una hora se abre un reloj, y al tocar el cupo, un selector. Los cambios se aplican al pulsar «Guardar».

Debajo, en «Vacaciones y días extra», el botón «Añadir» abre un diálogo con dos opciones (figura II.13):

- **Vacaciones**: se elige un día o un rango de días en el calendario y quedan cerrados.
- **Día extra**: se elige un día, su horario y su cupo. Sirve para abrir un día que normalmente está cerrado o para cambiar el horario de un día concreto.

Las vacaciones y los días extra aparecen en una lista (figura II.14), mandan sobre la plantilla semanal y se pueden borrar con la cruz de cada fila.

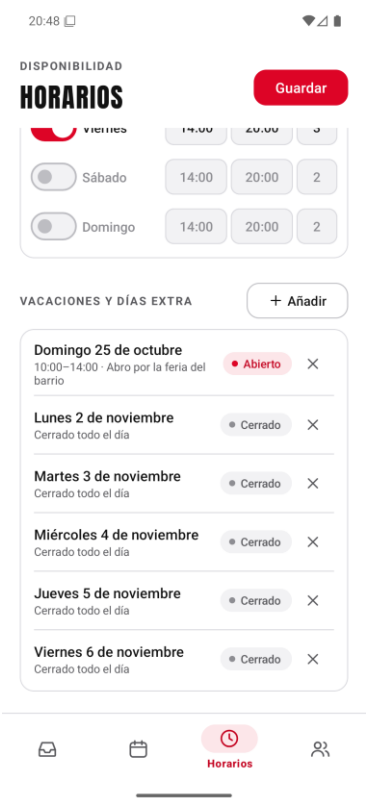
Figura II.12
Horarios



Figura II.13
Día extra



Figura II.14
Vacaciones y días extra guardados



Nota. Las figuras II.12 a II.14 son de elaboración propia.

Clientes (figuras II.15 y II.16). En «Invitar a un cliente» se escribe una nota opcional para recordar a quién va dirigido el código, por ejemplo «Primo de Dani», y se pulsa «Generar». El código aparece con un botón para copiarlo y enviarlo al cliente por el medio que se prefiera. Debajo se listan las invitaciones con su estado (pendiente, usada o caducada), y en las usadas aparece qué @usuario se registró con ella. Al final están los clientes registrados, que se pueden aprobar o bloquear. Un cliente bloqueado no puede pedir citas, pero las que ya tenía no se borran.

Figura II.15
Clientes



Figura II.16
Código recién generado



Nota. Las figuras II.15 y II.16 son de elaboración propia.

Anexo III. Documentación técnica de diseño

Este anexo recoge el material de diseño que no cabe en el cuerpo de la memoria. Incluye la descripción completa de los diez casos de uso y su trazabilidad con pantallas, rutas y requisitos, los diagramas de clases y de secuencia completos, los mockups que no aparecen en la figura 1, el diagrama de estados de una cita y tres tablas de detalle (base de datos, seguridad y módulos del ciclo).

Los códigos de error que aparecen en los casos de uso son los que devuelve la API en el campo `error` de la respuesta, con el código HTTP entre paréntesis. Las pantallas se nombran como en los mockups: A1 a A3 para el acceso, C1 a C3 para el cliente y D1 a D4 para el barbero. La D5, el diálogo de la cita a mano, es de la segunda fase y no tiene mockup.

III.1 Descripción de los casos de uso

Tabla III.1

Caso de uso UC01

UC01	**Registrarse con código**
Actor	Visitante
Precondición	Tiene un código de invitación vigente que le ha dado el barbero.
Flujo básico	1. Abre A2, lee el aviso de privacidad y escribe código, @usuario, teléfono y contraseña. 2. La app valida el formato mientras escribe. 3. Envía `POST /api/auth/registro`. 4. La API comprueba la invitación y crea el usuario con rol `cliente` en estado `pendiente`. 5. La API devuelve el token y los datos del usuario. 6. La app guarda la sesión y muestra A3.
Alternativos y errores	4a. Formato incorrecto: `datos_invalidos` (400). 4b. Código inexistente: `invitacion_invalida` (400). Ya usado: `invitacion_usada` (409). Pasado de fecha: `invitacion_caducada` (409). 4c. @usuario o teléfono ya registrados: `duplicado` (409), y la invitación no se gasta. 4d. Demasiados intentos seguidos: `demasiados_intentos` (429).
Postcondición	Existe un cliente `pendiente`, la invitación queda `usada` con el nuevo usuario en `usada_por` y hay una sesión iniciada.

Nota. Elaboración propia.

Tabla III.2

Caso de uso UC02

UC02	**Iniciar sesión**
Actor	Cliente o barbero (usuario registrado)

UC02	Iniciar sesión
Precondición	Tiene cuenta, en cualquier estado.
Flujo básico	1. Abre A1. Si hay un token guardado y `GET /api/yo` lo acepta, este paso se salta. 2. Escribe @usuario y contraseña. 3. Envía `POST /api/auth/login`. 4. La API comprueba el hash de la contraseña y devuelve un token. 5. La app lo guarda cifrado y llama a `GET /api/yo`. 6. Según el rol y el estado, el barbero va a D1, el cliente aprobado a C1 y el cliente pendiente o bloqueado a A3, que muestra un texto distinto en cada caso.
Alternativos y errores	1a. Token guardado rechazado: `token_invalido` (401). La app lo borra y muestra A1. 3a. Campos vacíos: `datos_invalidos` (400). 3b. Usuario o contraseña incorrectos: `credenciales_invalidas` (401), con el mismo mensaje en los dos casos para no revelar qué usuarios existen. 3c. Demasiados intentos: `demasiados_intentos` (429).
Postcondición	La sesión queda activa. Para cerrarla, el cliente usa «Cerrar sesión» en C3 y el barbero el icono de salida de D4. En los dos casos se borra el token del móvil sin llamar a la API.

Nota. Elaboración propia.

Tabla III.3

Caso de uso UC03

UC03	Consultar disponibilidad
Actor	Cliente
Precondición	Sesión iniciada y cuenta aprobada.
Flujo básico	1. En C2 la app pide `GET /api/dias`. 2. Pinta los 14 días del horizonte como libre, completo o vacaciones. Los cerrados quedan deshabilitados. 3. El cliente elige un día libre. 4. La app pide `GET /api/huecos?fecha=…`. 5. Muestra las horas del día y deshabilita las ocupadas.
Alternativos y errores	1a. Cliente pendiente o bloqueado: `no_aprobado` (403). La app muestra A3. 1b. Sesión caducada: `token_invalido` (401). La app vuelve a A1. 4a. Fecha mal escrita: `datos_invalidos` (400). Fuera del horizonte: `fecha_fuera_de_horizonte` (400). *. Sin conexión, la app muestra un mensaje y deja reintentar.
Postcondición	No cambia ningún dato. El cliente sabe qué días y horas están libres.

Nota. Elaboración propia.

Tabla III.4

Caso de uso UC04

UC04	Solicitar cita
Actor	Cliente
Precondición	Cuenta aprobada y un hueco libre elegido en UC03.

UC04	Solicitar cita
Flujo básico	1. En C2 pulsa el botón «Pedir cita», que repite el día y la hora. 2. La app envía `POST /api/citas` con `fecha` y `hora_pedida`. 3. La API comprueba horizonte, día abierto, tramo, hora en punto, hora no pasada, cupo y hueco, en ese orden. 4. Guarda la cita en estado `pendiente`. 5. Responde 201 con la cita. 6. La app avisa de que la solicitud se ha enviado y la cita aparece como pendiente en C1.
Alternativos y errores	3a. `fecha_fuera_de_horizonte`, `hora_fuera_de_tramo`, `hora_invalida` o `hora_pasada` (400). 3b. `dia_cerrado`, `dia_lleno` o `hueco_ocupado` (409). La app vuelve a cargar los huecos. 3c. Formato incorrecto: `datos_invalidos` (400). 3d. Cliente no aprobado: `no_aprobado` (403).
Postcondición	Hay una cita nueva `pendiente` que ocupa su hora y cuenta para el cupo hasta que el barbero decida.

Nota. Elaboración propia.

Tabla III.5

Caso de uso UC05

UC05	Gestionar mis citas (ver y cancelar)
Actor	Cliente
Precondición	Cuenta aprobada.
Flujo básico	1. En C1 la app pide `GET /api/mis-citas`. 2. Muestra una tarjeta por cita con el día, la hora y el estado escrito, sin precio ni propina. 3. En una cita pendiente o confirmada, el cliente pulsa cancelar. 4. La app pide confirmación en un diálogo. 5. Envía `POST /api/citas/{id}/cancelar`. 6. La cita pasa a `cancelada` y la lista se recarga.
Alternativos y errores	5a. Cita de otro cliente o inexistente: `no_encontrado` (404). 5b. La cita ya no está activa: `estado_invalido` (409). 5c. La hora de la cita ya ha pasado: `cita_pasada` (409). 5d. `no_aprobado` (403) o `token_invalido` (401), igual que en UC03.
Postcondición	La cita queda `cancelada` y su hora y su plaza del cupo se liberan en ese momento.

Nota. Elaboración propia.

Tabla III.6

Caso de uso UC06

UC06	Generar invitación
Actor	Barbero
Precondición	Sesión de barbero.

UC06	Generar invitación
Flujo básico	1. En D4 escribe una nota opcional y pulsa «Generar». 2. La app envía `POST /api/admin/invitaciones`. 3. La API crea un código aleatorio de 12 caracteres, válido 7 días y de un solo uso. 4. La app muestra el código con un botón para copiarlo. 5. D4 recarga la lista con `GET /api/admin/invitaciones`. Cada fila lleva el código, la nota y el estado (pendiente, usada o caducada), y la nueva aparece como pendiente. «Caducada» no se guarda en la base de datos, sino que la API la calcula para las pendientes cuya fecha de caducidad ya ha pasado.
Alternativos y errores	2a. Nota de más de 60 caracteres: `datos_invalidos` (400). 2b. Sesión de cliente: `sin_permiso` (403). 2c. Sesión caducada: `token_invalido` (401).
Postcondición	Existe una invitación `pendiente` creada por el barbero que caduca a los 7 días.

Nota. Elaboración propia.

Tabla III.7

Caso de uso UC07

UC07	Aprobar o bloquear cliente
Actor	Barbero
Precondición	Sesión de barbero y al menos un cliente registrado.
Flujo básico	1. En D4 la app pide `GET /api/admin/clientes`. 2. El barbero elige un cliente y pulsa aprobar o bloquear. 3. La app envía `POST /api/admin/clientes/{id}/aprobar` o `POST /api/admin/clientes/{id}/bloquear`. 4. La API cambia el estado y, si aprueba, guarda la fecha en `aprobado_en`. 5. La lista se actualiza.
Alternativos y errores	3a. Cliente inexistente: `no_encontrado` (404). 3b. Ya estaba en ese estado: `estado_invalido` (409). 3c. Sesión de cliente: `sin_permiso` (403).
Postcondición	El nuevo estado se aplica desde la siguiente petición del cliente. Sus citas no se tocan.

Nota. Elaboración propia.

Tabla III.8

Caso de uso UC08

UC08	Gestionar solicitudes (confirmar o rechazar)
Actor	Barbero
Precondición	Hay citas pendientes.
Flujo básico	1. En D1 la app pide `GET /api/admin/citas?estado=pendiente`. 2. Cada tarjeta muestra el @usuario, el día y la hora pedida. 3. El barbero deja la hora pedida o elige otra hora final del mismo día. 4. Confirma con `POST /api/admin/citas/{id}/confirmar` o, tras un diálogo, rechaza con `POST /api/admin/citas/{id}/rechazar`. 5. La cita pasa a `confirmada` o `rechazada` y desaparece de la lista.

UC08	Gestionar solicitudes (confirmar o rechazar)
Alternativos y errores	4a. Otra cita activa a esa hora: `hueco_ocupado` (409). 4b. La cita ya no está pendiente: `estado_invalido` (409). Su hora ya ha pasado: `cita_pasada` (409). 4c. Hora final con formato o rango incorrectos: `datos_invalidos` (400). 4d. Cita inexistente: `no_encontrado` (404).
Postcondición	La cita queda `confirmada` con su `hora_final` y la fecha de la decisión, o `rechazada`, que libera la hora y la plaza del cupo.

Nota. Elaboración propia.

Tabla III.9

Caso de uso UC09

UC09	Gestionar la agenda (asistencia y cancelación)
Actor	Barbero
Precondición	Hay citas confirmadas.
Flujo básico	1. En D2 la app pide las citas confirmadas de la semana visible con `GET /api/admin/citas?estado=confirmada&desde=…&hasta=….` 2. El barbero toca un día y elige una cita. 3. Si su hora ya ha pasado, pulsa «Asistió» o «No vino», que envía `POST /api/admin/citas/{id}/asistencia` con el `resultado`. 4. Si es futura, pulsa cancelar y lo confirma en el diálogo, que envía `POST /api/admin/citas/{id}/cancelar`. 5. La cita deja de estar confirmada y sale de la agenda al recargar.
Alternativos y errores	3a. Asistencia antes de la hora de la cita: `cita_futura` (409). 3b. `resultado` desconocido: `datos_invalidos` (400). 4a. Cancelar una cita que ya ha pasado: `cita_pasada` (409). *. La cita no está confirmada: `estado_invalido` (409). Cita inexistente: `no_encontrado` (404).
Postcondición	La cita queda `hecha` (con el precio del corte del cliente copiado en ese momento), `no_asistio` o `cancelada`. Solo la cancelación libera hora y cupo.

Nota. Elaboración propia.

Tabla III.10

Caso de uso UC10

UC10	Configurar horarios y vacaciones
Actor	Barbero
Precondición	Sesión de barbero.
Flujo básico	1. En D3 la app pide `GET /api/admin/horarios` y `GET /api/admin/excepciones`. 2. Para cada día de la semana fija si abre, la hora de inicio, la de fin y el cupo, y guarda con `PUT /api/admin/horarios`. 3. Con «Añadir» crea una excepción. Unas vacaciones cierran uno o varios días seguidos, y un día extra abre una fecha concreta con su propio tramo y cupo. Se guarda con `POST /api/admin/excepciones`. 4. Para quitar una excepción pulsa la cruz de su fila, que envía `DELETE /api/admin/excepciones/{id}`. 5. Los cambios se aplican desde ese momento en UC03 y UC04.

UC10	Configurar horarios y vacaciones
Alternativos y errores	2a. Inicio igual o posterior al fin, día de la semana fuera de 1 a 7, cupo fuera de rango, fecha pasada o vacaciones con horas: `datos_invalidos` (400). 4a. Excepción inexistente: `no_encontrado` (404). *. Sesión de cliente: `sin_permiso` (403).
Postcondición	El horario y las excepciones quedan actualizados. Una excepción manda sobre el horario semanal en su fecha, y las citas que ya existían no cambian.

Nota. Elaboración propia.

III.2 Trazabilidad

La tabla III.11 relaciona cada caso de uso con la pantalla donde ocurre, las rutas de la API que usa y los requisitos funcionales que cubre. Así se comprueba que no queda ningún requisito sin caso de uso ni ninguna ruta sin pantalla, salvo `/api/salud`, que solo sirve para comprobar la instalación.

Tabla III.11

Trazabilidad entre casos de uso, pantallas, rutas y requisitos

Caso de uso	Pantalla	Rutas de la API	Requisitos
UC01 Registrarse con código	A2 y luego A3	`POST /api/auth/registro`, `GET /api/yo`	RF-01, RF-02, RF-03, RF-04, RF-08
UC02 Iniciar sesión	A1 (C3 y D4 para salir)	`POST /api/auth/login`, `GET /api/yo`	RF-04, RF-05, RF-06, RF-07, RF-08
UC03 Consultar disponibilidad	C2	`GET /api/dias`, `GET /api/huecos`	RF-09, RF-10, RF-11
UC04 Solicitar cita	C2	`POST /api/citas`	RF-12, RF-13, RF-14
UC05 Gestionar mis citas	C1	`GET /api/mis-citas`, `POST /api/citas/{id}/cancelar`	RF-15, RF-16
UC06 Generar invitación	D4	`POST /api/admin/invitaciones`, `GET /api/admin/invitaciones`	RF-17, RF-18
UC07 Aprobar o bloquear cliente	D4	`GET /api/admin/clientes`, `POST /api/admin/clientes/{id}/aprobar`, `POST /api/admin/clientes/{id}/bloquear`	RF-08, RF-19, RF-20, RF-21
UC08 Gestionar solicitudes	D1	`GET /api/admin/citas?estado=pendiente`, `POST /api/admin/citas/{id}/confirmar`, `POST /api/admin/citas/{id}/rechazar`	RF-22, RF-23, RF-24

Caso de uso	Pantalla	Rutas de la API	Requisitos
UC09 Gestionar la agenda	D2	`GET /api/admin/citas?estado=confirmada`, `POST /api/admin/citas/{id}/asistencia`, `POST /api/admin/citas/{id}/cancelar`	RF-25, RF-26, RF-27
UC10 Configurar horarios y vacaciones	D3	`GET` y `PUT /api/admin/horarios`, `GET` y `POST /api/admin/excepciones`, `DELETE /api/admin/excepciones/{id}`	RF-11, RF-28, RF-29
Cita a mano (P2, ligada a UC08 y UC09)	D5, diálogo sobre D2	`POST /api/admin/citas`	RF-P2-02
Sin caso de uso	—	`GET /api/salud` (anexo I)	RF-30

Nota. Elaboración propia.

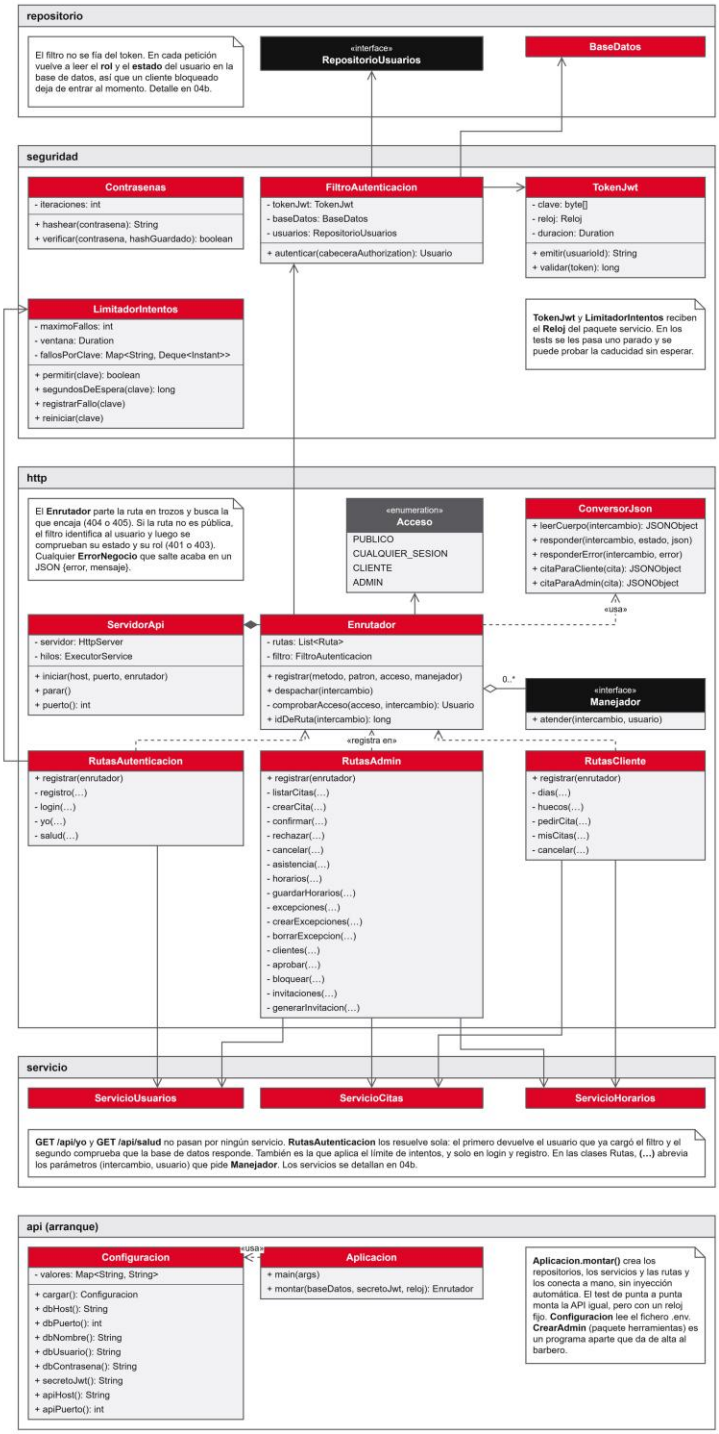
C3 no tiene caso de uso propio, porque cerrar sesión forma parte de UC02 y el cambio de tema es un requisito no funcional de usabilidad. La cita a mano tampoco lo tiene, ya que crea directamente una cita confirmada que después se lleva desde la agenda como cualquier otra. El requisito RF-25 preveía una vista de agenda por semana o por mes, pero solo se construyó la semanal.

III.3 Diagramas de clases

La figura 6 de la memoria muestra el modelo de la API. Los seis diagramas siguientes completan el resto de capas y se generaron a partir de las clases compiladas, de modo que los nombres coinciden con el código. Cada uno agrupa las clases por paquete.

Figura III.1

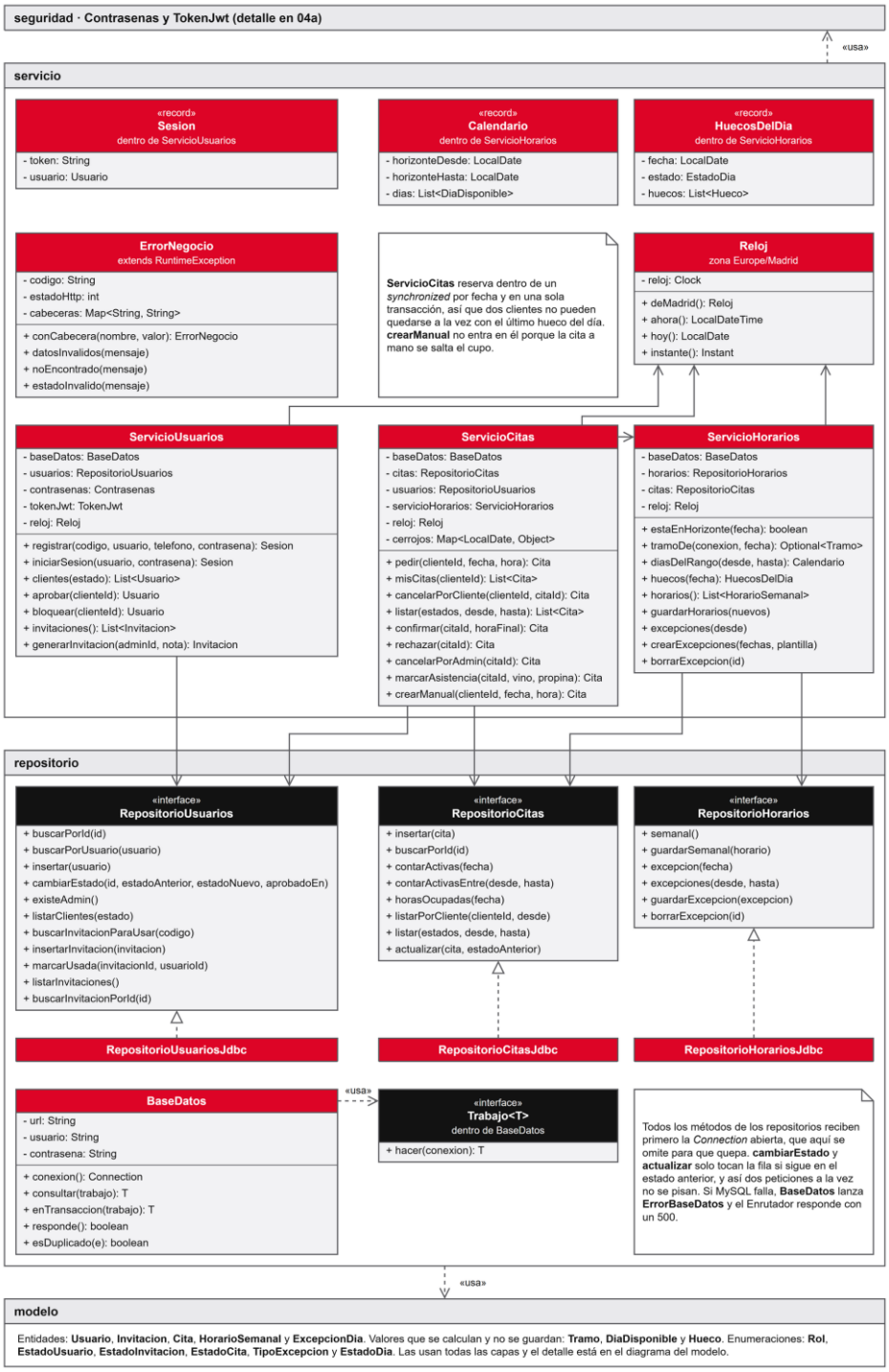
API: arranque, capa HTTP con el enrutador y las rutas, y seguridad (contraseñas, token JWT, filtro y limitador de intentos)



Nota. Elaboración propia.

Figura III.2

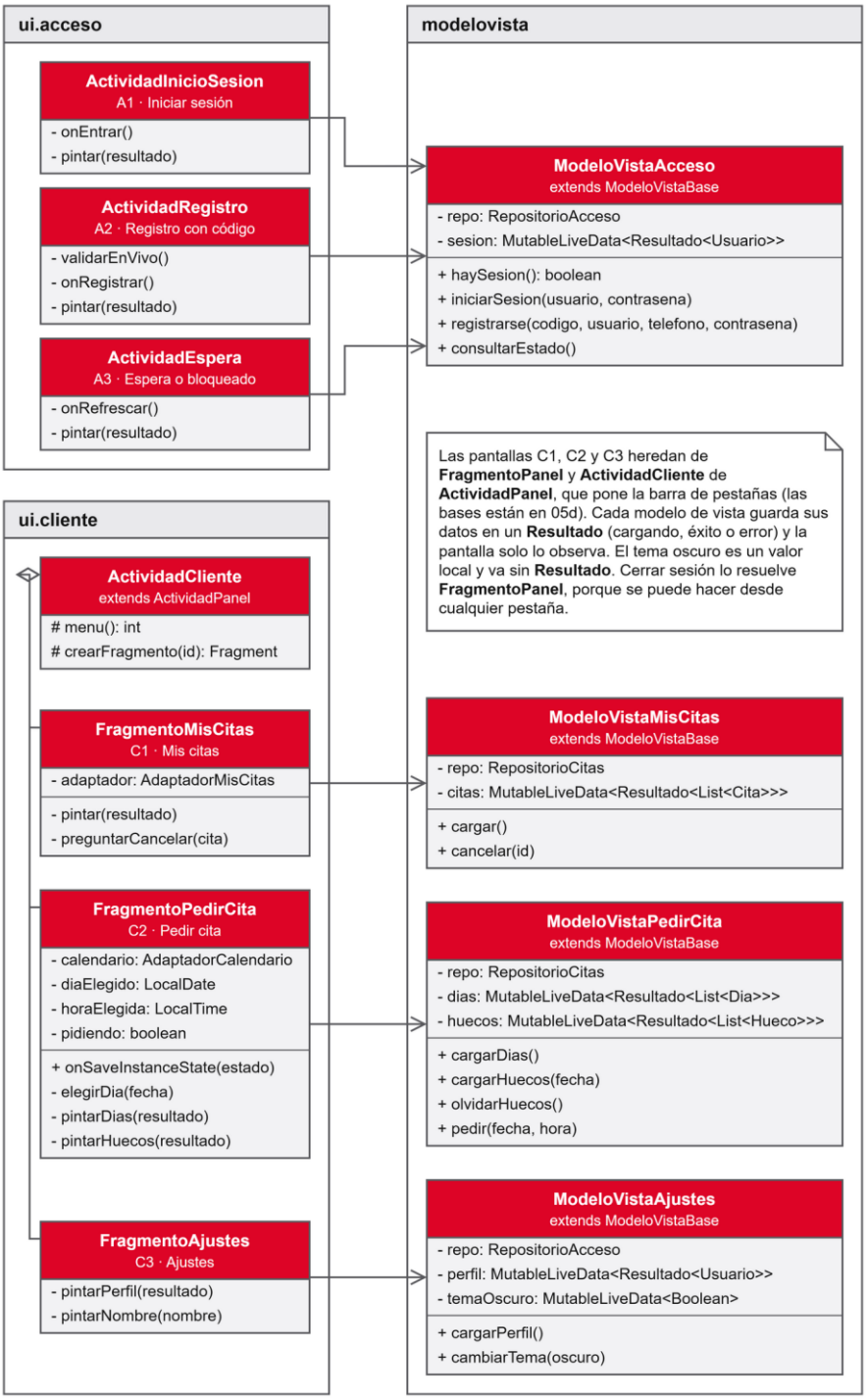
API: servicios de negocio, errores y repositorios



Nota. Elaboración propia.

Figura III.3

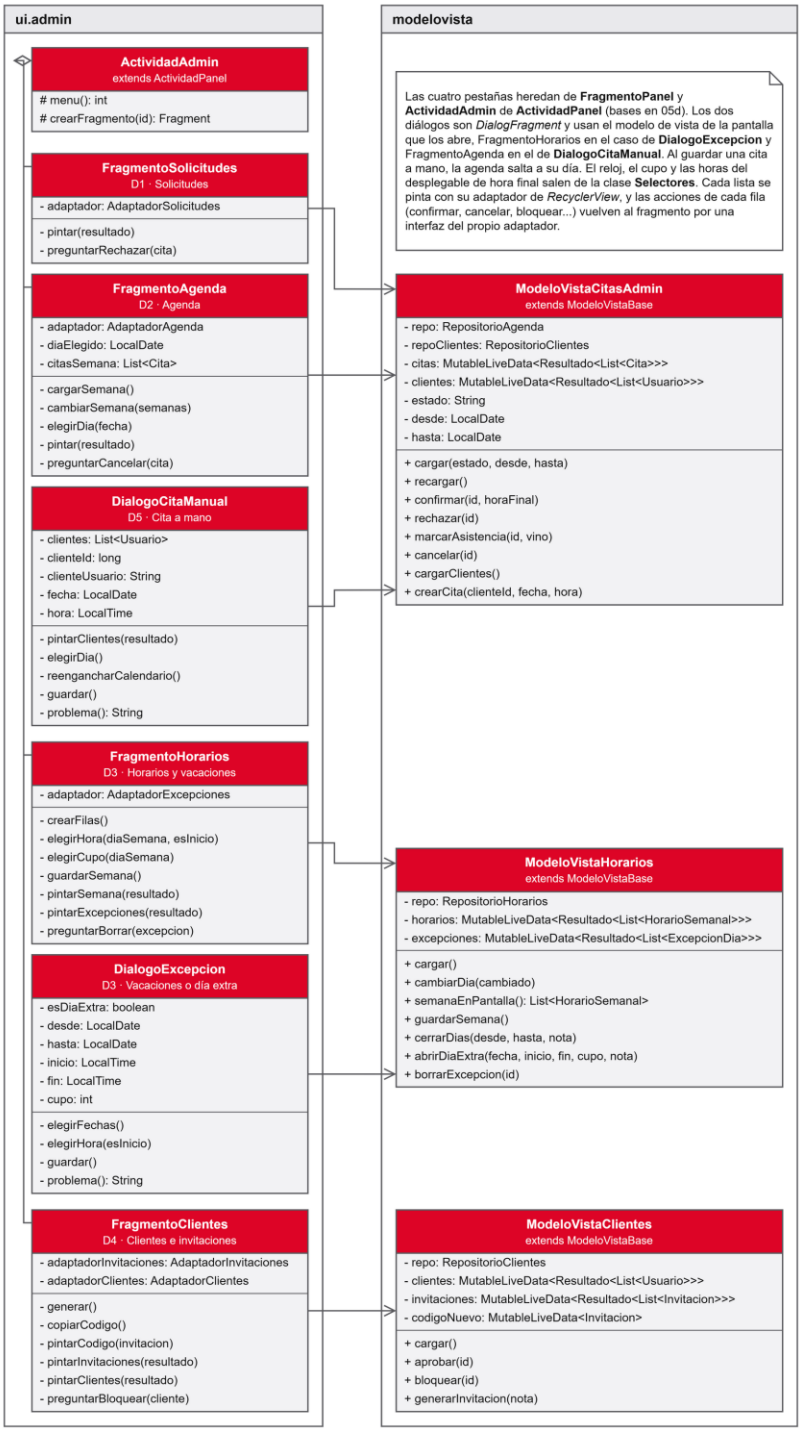
App: pantallas de acceso y del cliente con sus ModeloVista



Nota. Elaboración propia.

Figura III.4

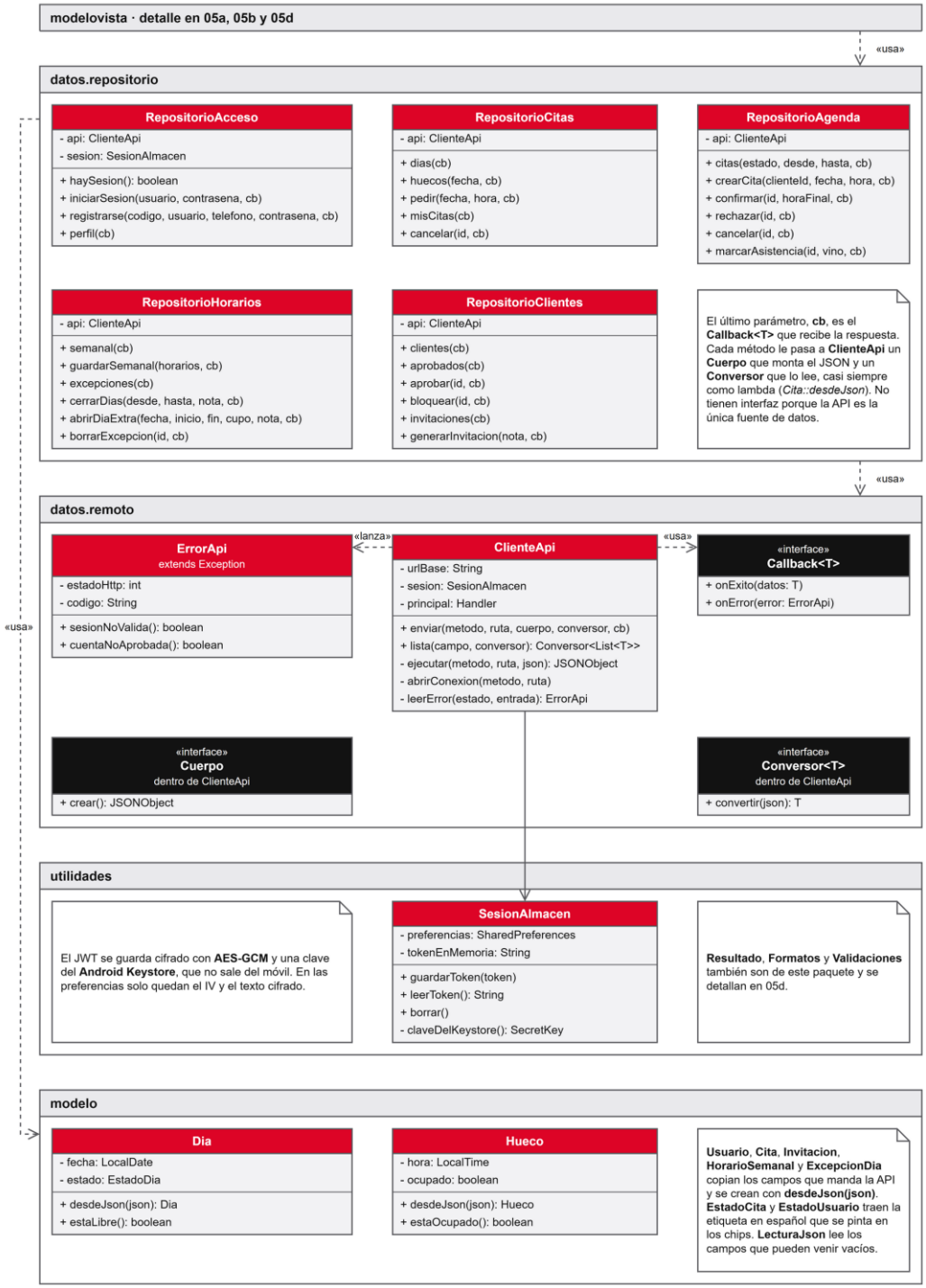
App: pantallas del barbero con sus ModeloVista



Nota. Elaboración propia.

Figura III.5

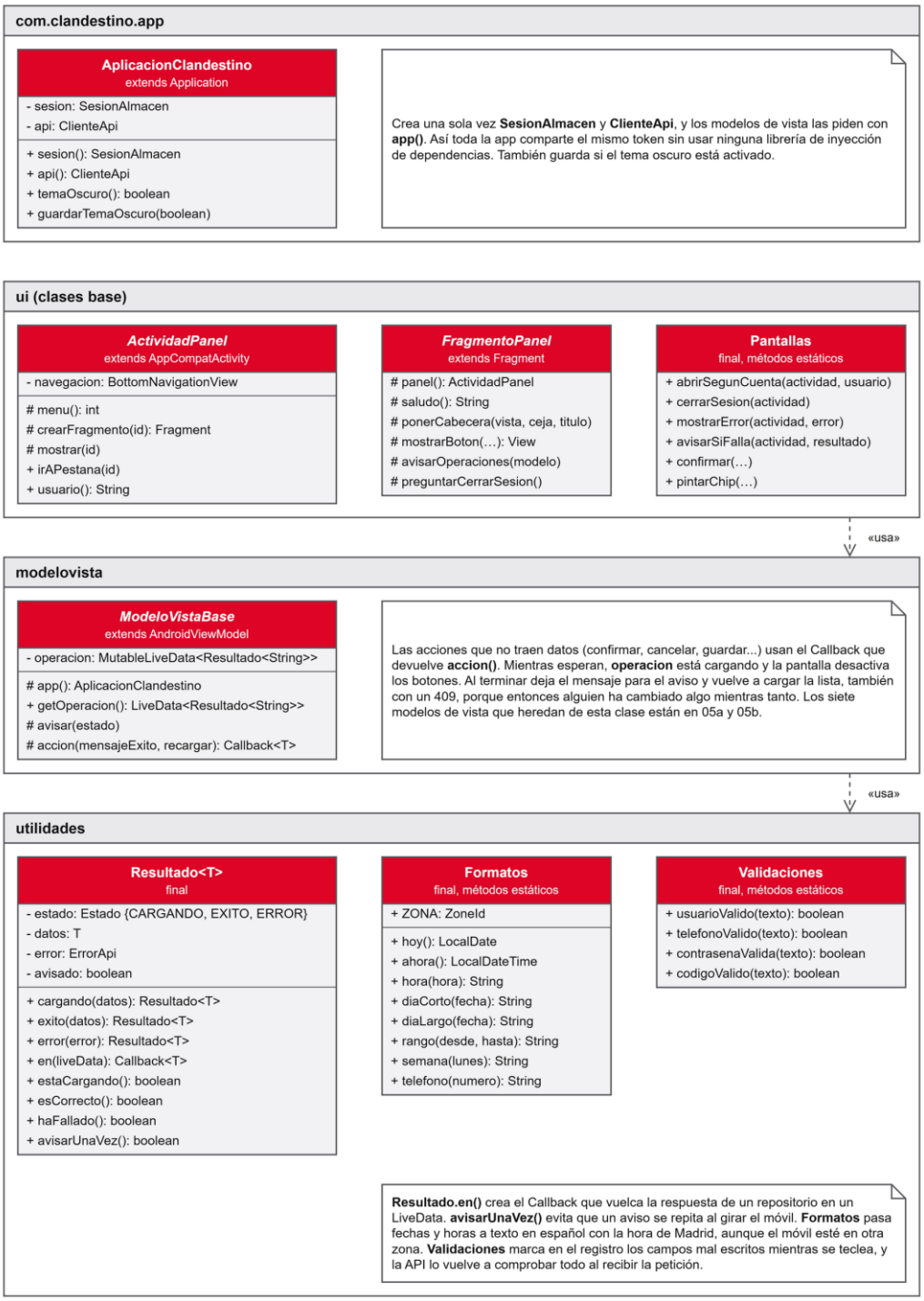
App: repositorios, cliente HTTP y almacén cifrado de la sesión



Nota. Elaboración propia.

Figura III.6

App: clase Application y clases base de actividades y fragmentos



Nota. Elaboración propia.

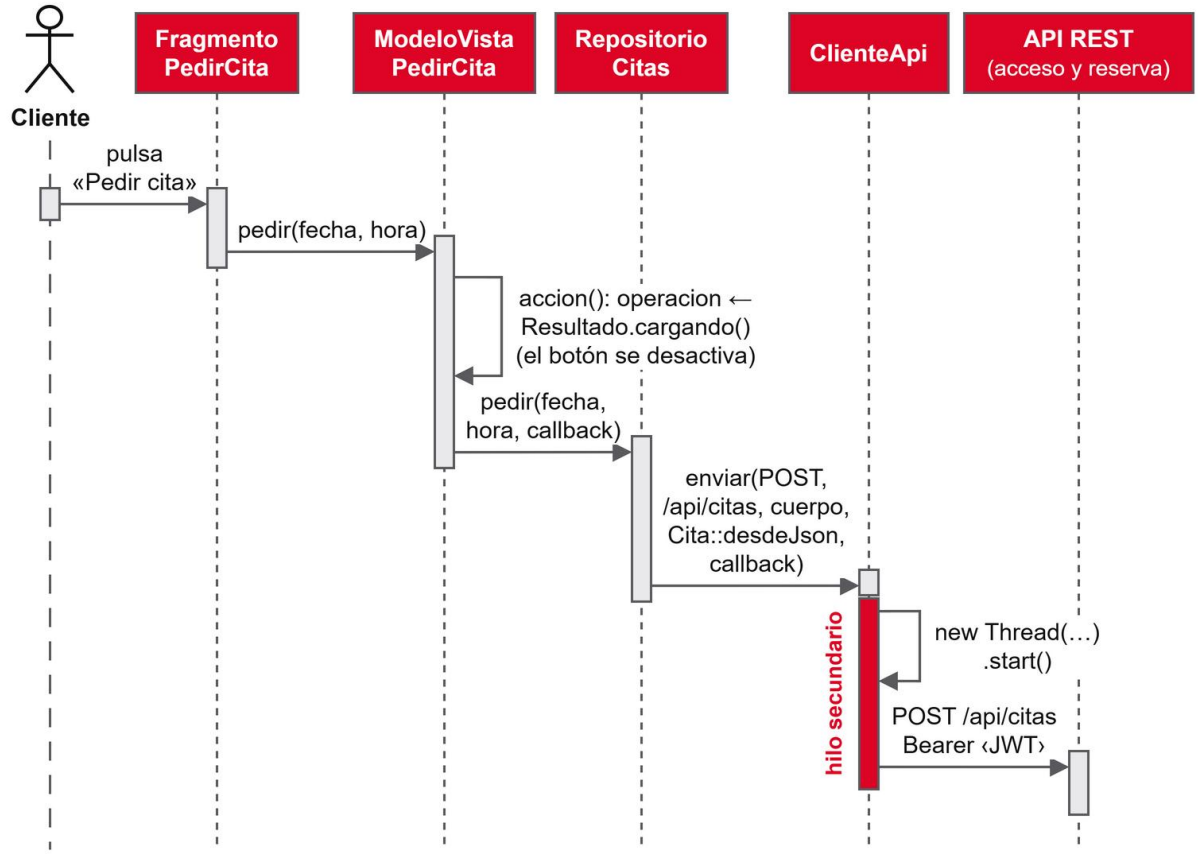
III.4 Diagramas de secuencia de «Solicitar cita»

El caso UC04 es el que atraviesa todas las capas, así que se dibujó entero, desde que el cliente pulsa el botón hasta que la cita queda guardada. Por tamaño se

partió en cinco diagramas, y el último es la inserción con el cerrojo por fecha que explica el apartado 7.4.

Figura III.7

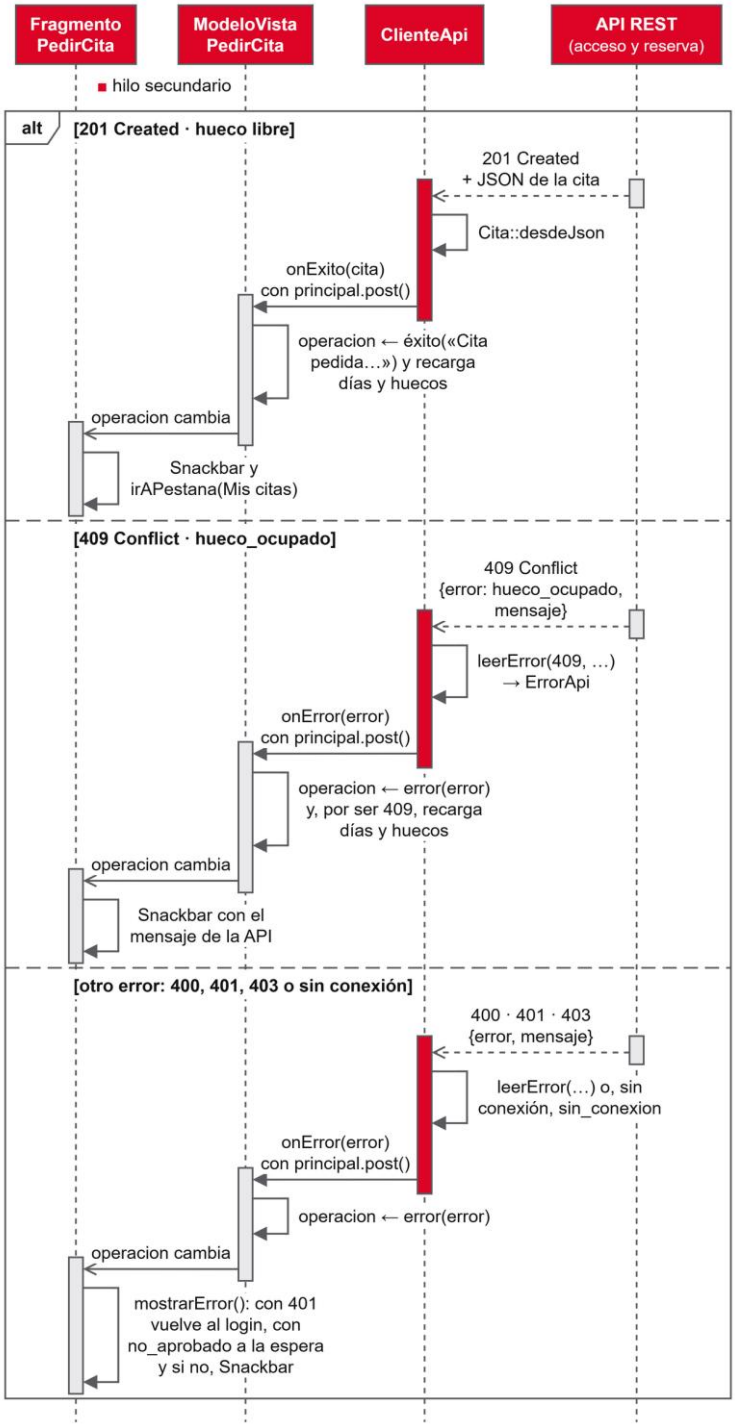
App, envío



Nota. El fragmento llama al ModeloVista, que pasa la petición al repositorio y este la lanza en un hilo aparte. Elaboración propia.

Figura III.8

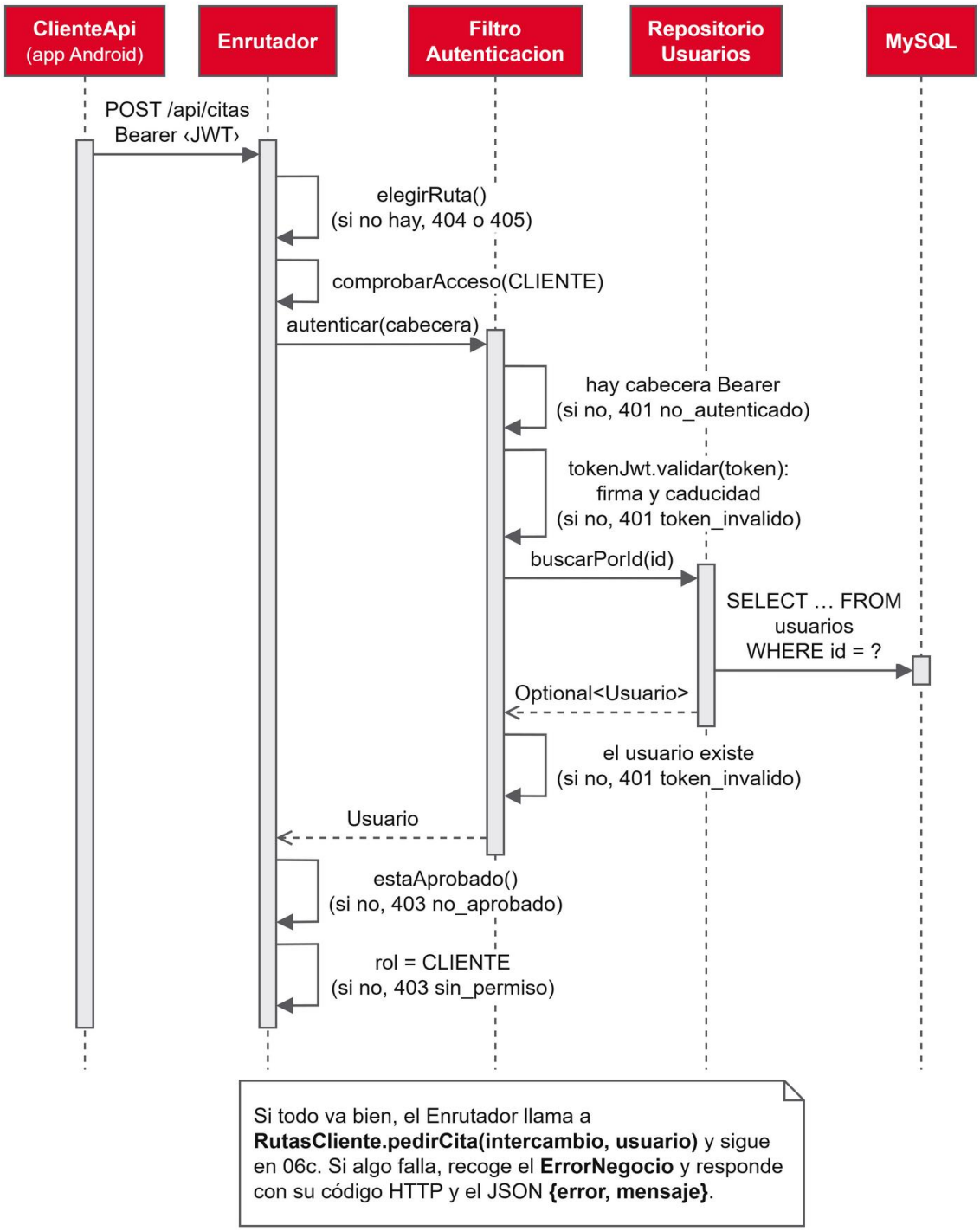
App, respuesta



Nota. El resultado vuelve al hilo principal y el fragmento reacciona de tres formas: cita creada, hora ya ocupada (recarga los huecos) u otro error. Elaboración propia.

Figura III.9

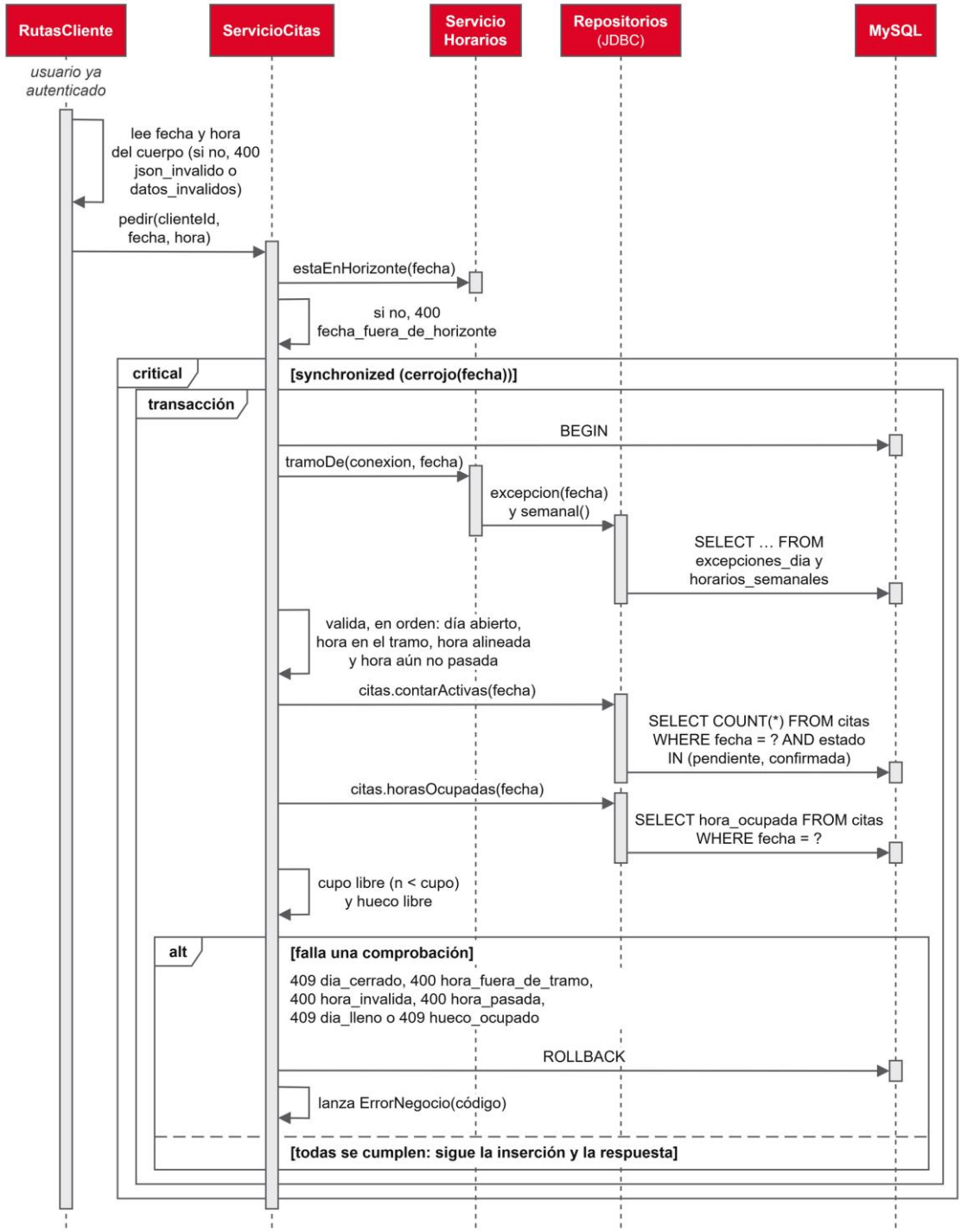
API, acceso



Nota. El enrutador elige la ruta, el filtro valida el token y busca al usuario, y el enrutador comprueba que es un cliente aprobado. Elaboración propia.

Figura III.10

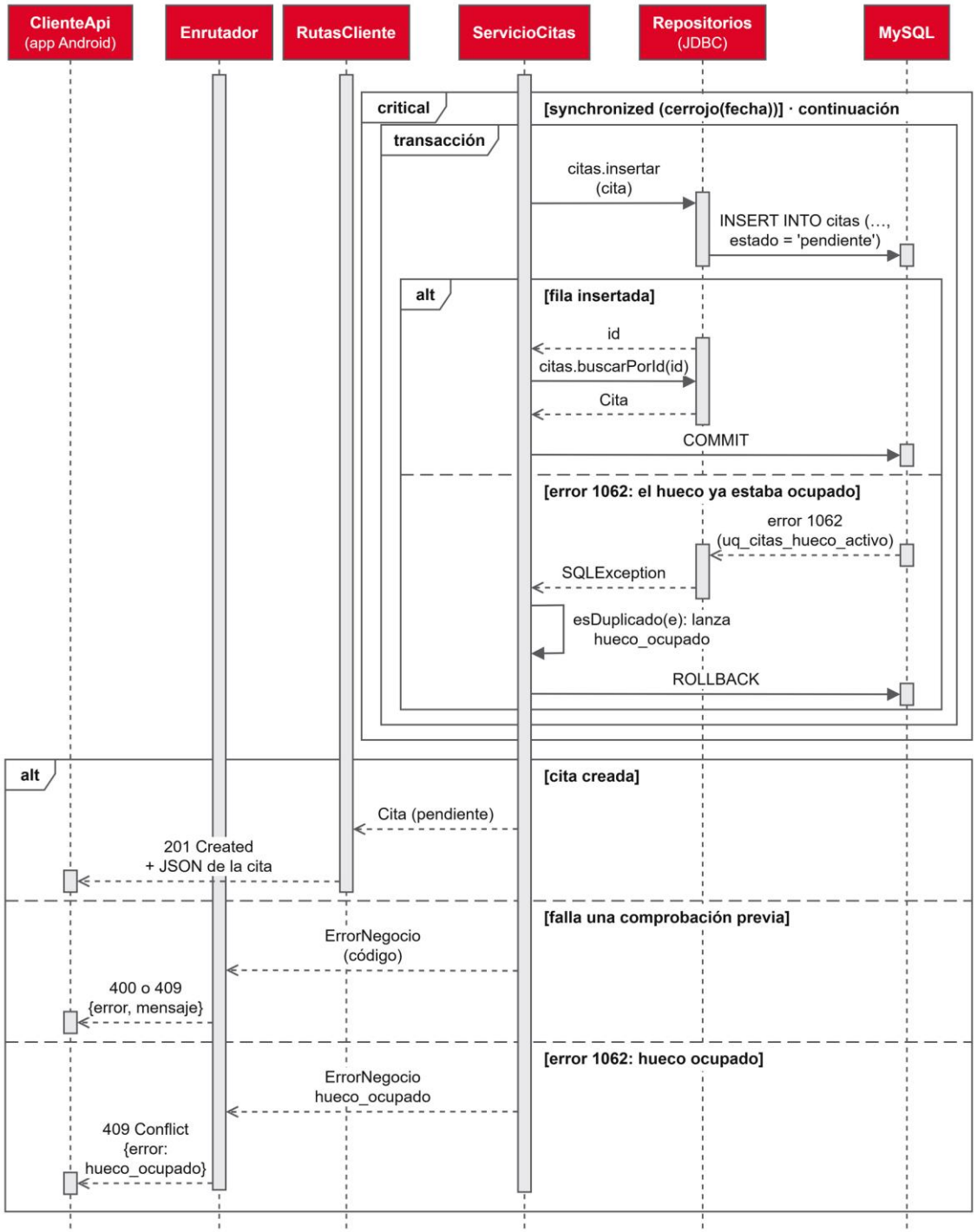
API, validación



Nota. Dentro del cerrojo de la fecha y de la transacción, el servicio comprueba horizonte, tramo, hora, cupo y hueco antes de reservar. Elaboración propia.

Figura III.11

API, inserción



Nota. El servicio cuenta las citas del día y guarda la nueva dentro del cerrojo de la fecha, y si MySQL devuelve el error 1062 la API responde 409 hueco_ocupado. Elaboración propia.

III.5 Mockups restantes

La figura 1 de la memoria reúne seis de los once mockups. Estos son los otros cinco: el registro, las dos variantes de la pantalla de espera, los ajustes y la pantalla de clientes e invitaciones.

Figura III.12
A2, registro con código

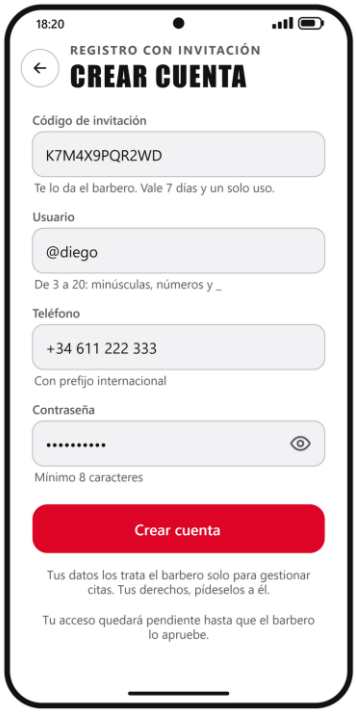


Figura III.13
A3, cuenta pendiente

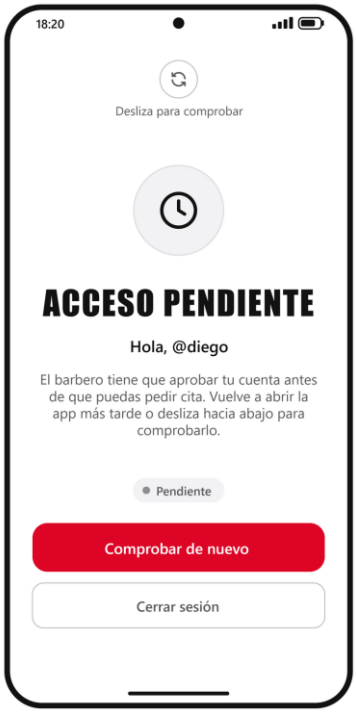


Figura III.14
A3, cuenta bloqueada

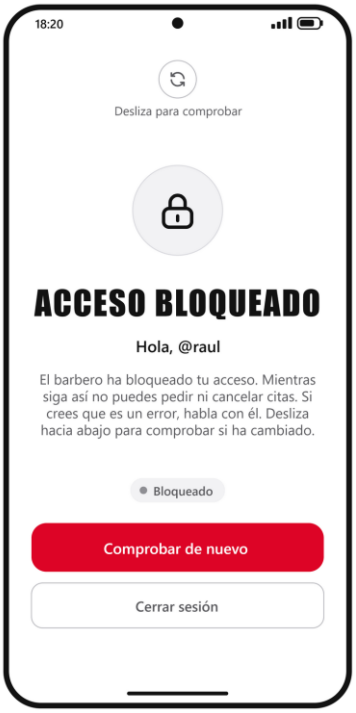


Figura III.15
C3, ajustes



Figura III.16
D4, clientes e invitaciones



Nota. Las figuras III.12 a III.16 son de elaboración propia.

La app final respeta los mockups casi por completo. El cambio más visible está en «Pedir cita» (figuras III.17 y III.18). El mockup mostraba un mes entero con flechas para avanzar, pero el horizonte de reserva quedó fijado en dos semanas y el calendario enseña solo esos días, sin flechas.

Figura III.17
C2, mockup de «Pedir cita»

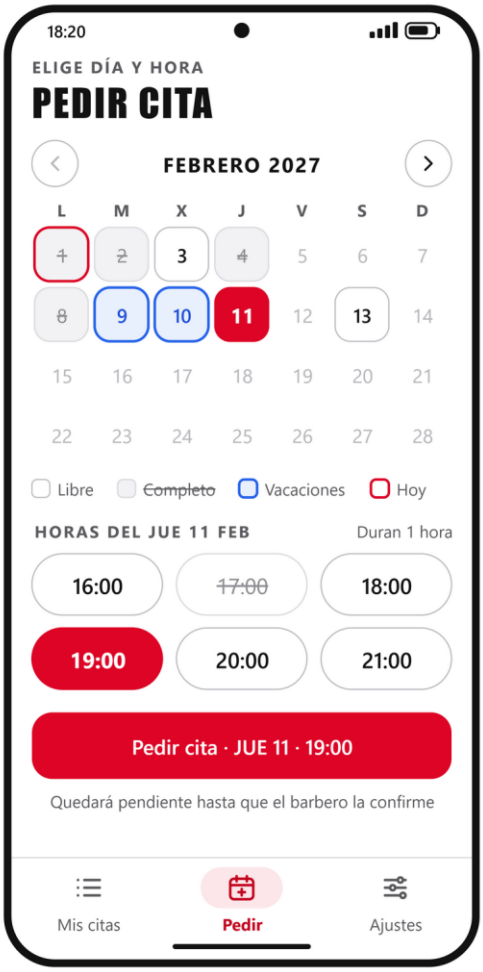
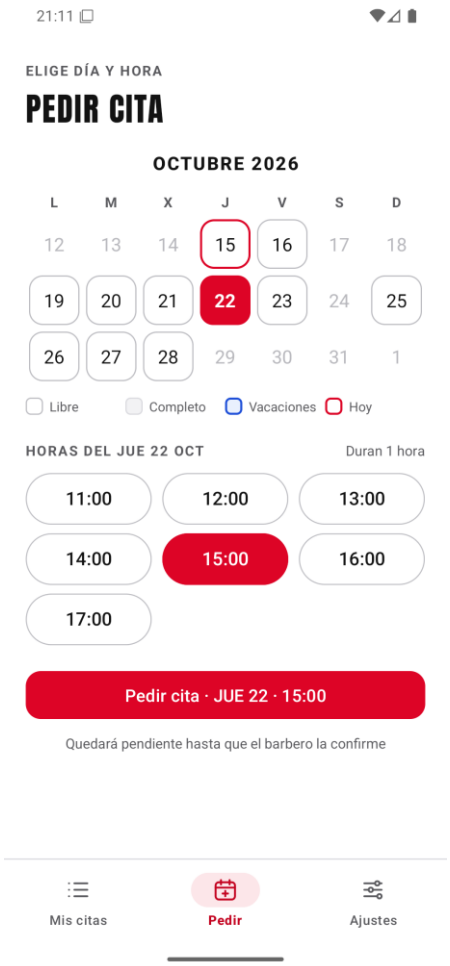


Figura III.18
C2, la app en el emulador



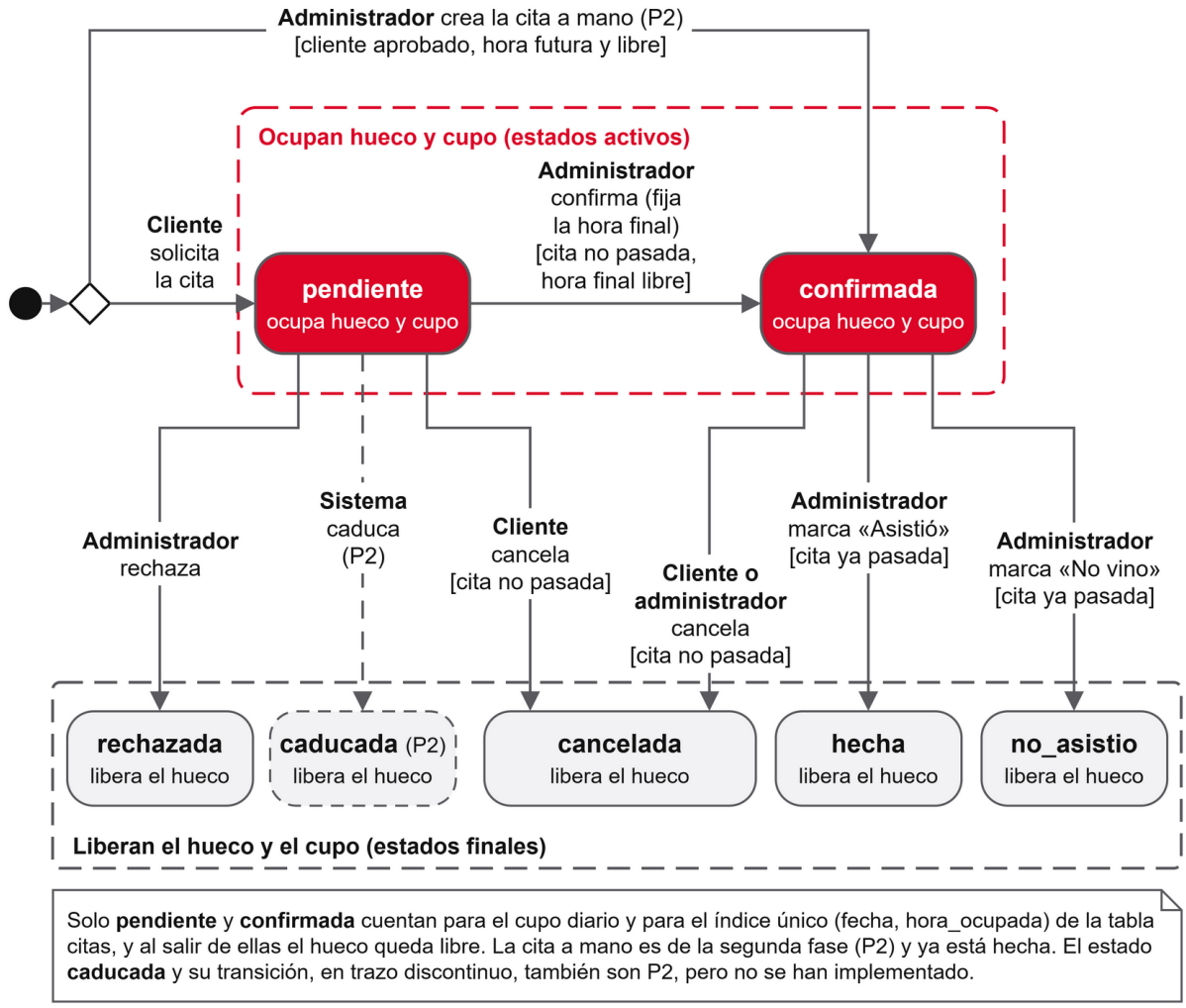
Nota. Las figuras III.17 y III.18 son de elaboración propia.

III.6 Estados de una cita

Una cita pasa por varios estados a lo largo de su vida (figura III.19). Solo las pendientes y las confirmadas ocupan hueco y cuentan para el cupo del día. El estado «caducada», que en la aplicación original marca las solicitudes que nadie respondió a tiempo, existe en la base de datos pero no lo asigna ningún proceso del prototipo, y el barrido periódico que lo haría queda para una segunda fase.

Figura III.19

Diagrama de estados de una cita



Nota. Elaboración propia.

III.7 Tablas de detalle

Tabla III.12

Tablas de la base de datos

Tabla	Qué guarda	Relaciones y restricciones principales
`usuarios`	Barbero y clientes: @usuario, teléfono, hash de la contraseña, rol, estado (pendiente, aprobado o bloqueado) y precio del corte	@usuario y teléfono únicos
`invitaciones`	Códigos de alta de un solo uso, con nota, estado y fecha de caducidad	`creada_por` y `usada_por` apuntan a `usuarios`
`citas`	Cada cita con su fecha, la hora pedida, la hora final que fija el barbero, el estado y el precio	`cliente_id` apunta a `usuarios`; índice único sobre la fecha y la hora ocupada

Tabla	Qué guarda	Relaciones y restricciones principales
`horarios_semanales`	Tramo de apertura y cupo de cada día de la semana	Un registro por día, del 1 (lunes) al 7 (domingo)
`excepciones_dia`	Fechas sueltas que mandan sobre el horario semanal: vacaciones (cerrado) o días extra (abierto, con su tramo y su cupo)	Una excepción por fecha; una restricción `CHECK` exige que un día cerrado no tenga horas

Nota. Elaboración propia.

Tabla III.13

Medidas de seguridad según las categorías de OWASP MASVS

Categoría MASVS	Qué hace Clandestino
STORAGE (almacenamiento)	El token de sesión se guarda cifrado con AES-GCM y una clave que vive en el Android Keystore y no puede extraerse del dispositivo (Android Developers, 2026a). La copia de seguridad automática de Android está desactivada (`allowBackup="false"`).
CRYPTO (criptografía)	Las contraseñas se guardan con PBKDF2-HMAC-SHA256, 600.000 iteraciones, sal aleatoria de 16 bytes y comparación en tiempo constante. El token se firma con HMAC-SHA256 y una clave de al menos 32 bytes.
AUTH (autenticación)	El rol y el estado se comprueban siempre en el servidor, nunca solo en la app. El token caduca a los 7 días. Tras 5 intentos fallidos en 15 minutos, la API responde 429. Un usuario inexistente tarda lo mismo en responder que una contraseña equivocada (775 ms medidos), así que por el tiempo no se puede saber qué cuentas existen.
NETWORK (red)	En la versión de depuración solo se permite HTTP sin cifrar hacia `10.0.2.2` y `localhost`, mediante la configuración de seguridad de red (Android Developers, 2026d). La versión de publicación no tiene esa excepción, pero el prototipo aún no tiene HTTPS (apartado 8.2 de la memoria).
CODE (código)	Todas las consultas usan `PreparedStatement`, lo que evita la inyección SQL. La API vuelve a validar cada dato aunque la app ya lo haya hecho y rechaza los cuerpos de más de 16 KiB.
PRIVACY (privacidad)	Solo se piden @usuario, teléfono y contraseña. El registro muestra un aviso de privacidad, como exige el RGPD (Reglamento (UE) 2016/679, 2016), y el cliente no ve los precios ni los datos de otros clientes.

Nota. Elaboración propia a partir de OWASP Mobile Application Security (2024).

Tabla III.14

Aplicación de los módulos del ciclo en el proyecto

Módulo	Dónde se aplica
Programación multimedia y dispositivos móviles	Actividades y su ciclo de vida (registrado con `Log.i`), fragmentos, `RecyclerView`, diálogos, `SharedPreferences`, `onSaveInstanceState` en «Pedir cita», `HttpURLConnection` en hilo secundario, Material 3
Programación de servicios y procesos	Servidor HTTP con grupo de hilos, `synchronized` en la sección crítica del cupo, prueba de hilos con `CountDownLatch`, hash de contraseñas y firma HMAC

Módulo	Dónde se aplica
Bases de datos	Modelo entidad-relación, restricciones, columna generada con índice único, transacciones, usuario sin privilegios de administración
Acceso a datos	JDBC con `PreparedStatement`, patrón repositorio, transacciones, JSON sobre HTTP
Programación	Programación orientada a objetos en Java: interfaces, `enum`, `record`, excepciones propias, colecciones
Desarrollo de interfaces	Mockups, layouts XML, tema claro y oscuro, validación en vivo, accesibilidad básica, manual de usuario
Entornos de desarrollo	Android Studio, Gradle, JUnit, UML con draw.io, lint
Empresa e iniciativa emprendedora	Análisis de mercado y propuesta de valor (apartado 3.4 de la memoria)

Nota. Elaboración propia.

Anexo IV. Casos de prueba manuales

Las pruebas automáticas (97 de la API, 26 de la app y 21 del esquema) se describen en el apartado 7.7. Este anexo recoge las que hice a mano: el recorrido completo de la app en el emulador, unas comprobaciones de la API con `curl` y la prueba de la cita a mano.

IV.1 Recorrido de la app en el emulador

Lo hice el 15 de octubre de 2026 en un emulador con Android 16 (API 36), pantalla de 1080 × 2400 y zona horaria Europe/Madrid, contra la API y la base de datos locales. Los casos van en el orden en que los ejecuté, porque cada uno parte del estado que deja el anterior. Las capturas del anexo II las tomé durante este recorrido, salvo las de la cita a mano (apartado IV.3) y la de la agenda, que repetí ese día para que saliera el botón «Nueva cita».

Tabla IV.1

Casos de prueba manuales en el emulador

Id	Caso	Pasos	Resultado esperado	Resultado
CP-01	Configurar horarios (UC10)	Entrar como barbero. En «Horarios», cambiar el tramo y el cupo de varios días y pulsar «Guardar». Añadir un día extra y unas vacaciones de varios días.	La plantilla se guarda. El día extra y las vacaciones aparecen en la lista, una fila por día de vacaciones.	Correcto (figuras II.12 a II.14)
CP-02	Generar invitación (UC06)	En «Clientes», escribir una nota y pulsar «Generar».	Aparece un código de 12 caracteres con el botón de copiar y la invitación sale como pendiente en la lista.	Correcto (figura II.16)
CP-03	Registrarse con código (UC01)	Cerrar sesión. En la entrada, pulsar «Crear cuenta con código» y rellenar el código del CP-02, el usuario `carlos_m`, un teléfono y una contraseña.	La cuenta se crea y la app muestra la pantalla de acceso pendiente. La invitación pasa a usada por `@carlos_m`.	Correcto (figuras II.2, II.3 y II.15)
CP-04	Aprobar cliente (UC07)	Entrar como barbero y, en «Clientes», aprobar a `carlos_m`. Después, volver a entrar como cliente.	El cliente aparece como aprobado y, al entrar, ve su panel en lugar de la pantalla de espera.	Correcto
CP-05	Solicitar cita (UC03 y UC04)	Como cliente, en «Pedir», tocar el jueves 22, elegir las 15:00 y pulsar el botón «Pedir cita».	La cita se crea y aparece como pendiente en «Mis citas».	Correcto (figuras II.4 y II.5)

Id	Caso	Pasos	Resultado esperado	Resultado
CP-06	Confirmar con otra hora (UC08 y UC09)	Como barbero, en «Solicitudes», cambiar la hora final a 15:30 y pulsar «Confirmar». Abrir «Agenda» en esa semana y tocar el jueves.	La solicitud desaparece de la lista y la cita sale confirmada a las 15:30 en la agenda.	Correcto (figuras II.7 y II.8)
CP-07	Tema oscuro y cerrar sesión	En «Ajustes», activar el tema oscuro y después pulsar «Cerrar sesión».	La app pasa a colores oscuros y, al cerrar sesión, vuelve a la pantalla de entrada.	Correcto

Nota. Elaboración propia.

Durante el recorrido aparecieron ocho fallos visuales y de configuración que las pruebas automáticas no podían detectar, como el texto de las horas descentrado en «Pedir cita» o el botón «No vino» que parecía activo en una cita futura. Se corrigieron en el momento y se repitió el paso afectado. Están descritos en la entrada del 15 de octubre del diario técnico.

IV.2 Comprobaciones de la API con `curl`

Las ejecuté el 16 de octubre de 2026 contra la API en marcha en `http://127.0.0.1:8080`. Sirven para ver de un vistazo que la API responde con un código HTTP correcto y un error con nombre fijo en los casos más habituales. Estas mismas situaciones también están cubiertas por la clase de prueba `PruebaApi`.

Tabla IV.2

Comprobaciones manuales de la API

Id	Petición	Resultado esperado	Respuesta obtenida
CP-08	`GET /api/salud`	200 y base de datos disponible	200 `{"estado":"ok","base_de_datos":"ok"}`
CP-09	`GET /api/mis-citas` sin cabecera `Authorization`	401	401 `no_autenticado`
CP-10	`GET /api/mis-citas` con un token inventado	401	401 `token_invalido`
CP-11	`GET /api/nada`	404	404 `ruta_no_encontrada`
CP-12	`DELETE /api/salud`	405	405 `metodo_no_permitido`
CP-13	`POST /api/auth/login` con el cuerpo `{"usuario":`	400	400 `json_invalido`
CP-14	`POST /api/auth/login` con un cuerpo de unos 20 KB	400, porque el límite es de 16 KiB	400 `cuerpo_demasiado_grande`

Id	Petición	Resultado esperado	Respuesta obtenida
CP-15	`POST /api/auth/login` con un usuario que no existe	401 con el mensaje genérico	401 `credenciales_invalidas`, «Usuario o contraseña incorrectos.»

Nota. Elaboración propia.

Por ejemplo, el CP-09 se lanza así:

```
curl -i http://127.0.0.1:8080/api/mis-citas
```

IV.3 Cita a mano en el emulador

La hice el 17 de octubre de 2026 en el mismo emulador y con los datos que dejó el recorrido del apartado IV.1. Ese día era sábado, y los sábados están cerrados en la plantilla de horarios.

Tabla IV.3

Casos de prueba de la cita a mano

Id	Caso	Pasos	Resultado esperado	Resultado
CP-16	Cita a mano sin datos (RF-P2-02)	Como barbero, en «Agenda», pulsar «Nueva cita» y después «Guardar» sin elegir nada.	El diálogo sigue abierto y avisa con «Elige el cliente, el día y la hora.».	Correcto
CP-17	Cita a mano en un día cerrado (RF-P2-02)	Elegir a `@carlos_m`, el sábado 24 de octubre y las 12:00. Girar la pantalla a horizontal y volver a vertical. Pulsar «Guardar» y entrar después como `carlos_m`.	Tras el giro siguen elegidos el cliente, el día y la hora, y las listas enseñan todas sus opciones. Al guardar, la agenda pasa a la semana del 19 al 25 con el sábado elegido, la cita sale confirmada a las 12:00 y aparece el aviso «Cita guardada.». El cliente la ve confirmada en «Mis citas».	Correcto (figuras II.9 y II.10)
CP-18	Hora ya cogida (RF-P2-02)	Como barbero, repetir el CP-17 con el mismo cliente, día y hora.	La app avisa con «Esa hora ya está cogida.», que es el mensaje del `409 hueco_ocupado`, y no se crea otra cita.	Correcto (figura II.11)

Nota. Elaboración propia.

Anexo V. Diario de desarrollo

Este diario resume los hitos que constan en el registro técnico. Las dificultades y sus soluciones se han contrastado con ese registro. El proyecto se ha compaginado con el trabajo, con unas 3 horas al día entre semana y entre 2 y 4 horas en total por fin de semana. Las horas por bloque son un reparto aproximado de esa disponibilidad, no un registro de tiempo medido. Los bloques agrupan tareas de varias sesiones; la fecha del hito no significa que todas sus horas se concentraran en ese día.

Inicio y definición del alcance · 30 de septiembre y 1 de octubre de 2026

Etapa: inicio. **Dedicación estimada:** 6 horas.

El trabajo partió de la versión web de Clandestino y de las necesidades de un barbero que trabaja desde casa. Se delimitó una primera versión con acceso por invitación, aprobación de clientes, solicitudes de cita, agenda y horarios. Las métricas y los avisos quedaron fuera para poder terminar el ciclo de reserva completo.

La preparación del entorno exigió comprobar la compatibilidad entre Android Studio, Gradle y el JDK. También hubo que ajustar el consumo de memoria para trabajar con MySQL y el emulador en el mismo equipo. MySQL se instaló de forma portable para arrancarlo solo durante las pruebas. El siguiente paso fue definir los datos y las reglas antes de empezar la API.

Base de datos y diseño · 1 de octubre de 2026

Etapa: diseño. **Dedicación estimada:** 14 horas.

Se prepararon el esquema de cinco tablas, los casos de uso, los requisitos, el contrato de la API y los mockups. Las decisiones sobre el horizonte de reserva, los estados de la cita y las invitaciones quedaron por escrito para que la app y el servidor siguieran las mismas reglas.

MySQL no permitía trasladar directamente el índice único parcial de la versión original. Se utilizó una columna generada que conserva la hora mientras la cita está activa y toma el valor nulo cuando deja de estarlo. Las 21 pruebas del esquema comprobaron las restricciones y la liberación del hueco al cancelar.

La revisión del diseño encontró diferencias entre documentos y pantallas, entre ellas un mockup que mostraba el precio al cliente. También hubo que dividir diagramas

grandes para mejorar su lectura. Estas correcciones se hicieron antes de programar; el diseño quedó como referencia para la API.

Convenciones y contraseñas · 9 de octubre de 2026

Etapa: desarrollo. **Dedicación estimada:** 4 horas.

Se unificaron los nombres de clases, métodos y paquetes en español y se preparó la clase de contraseñas como referencia de estilo. Se comprobó que una misma contraseña producía huellas distintas y que la verificación rechazaba claves incorrectas y datos dañados.

El coste de PBKDF2 se midió en el equipo de desarrollo. Con 600 000 rondas, el tiempo dependía de la carga del ordenador y de si la máquina virtual acababa de arrancar. Se mantuvo el valor previsto para el uso normal y se redujo en las pruebas para que estas no fueran innecesariamente lentas. Después se continuó con los servicios y las rutas de la API.

API y reservas simultáneas · hasta el 14 de octubre de 2026

Etapa: desarrollo. **Dedicación estimada:** 16 horas.

Se completaron las capas HTTP, de servicios y de acceso a MySQL, junto con el registro, el inicio de sesión y las operaciones de cliente y barbero. Al utilizar el servidor HTTP del JDK hubo que resolver las rutas, los errores y el reparto de peticiones entre hilos.

La dificultad principal fue proteger el cupo diario. El índice único impedía repetir una hora exacta, pero dos peticiones a horas distintas podían contar la misma plaza libre. La solución combinó un bloqueo por fecha con una transacción que comprueba el cupo e inserta la cita antes de liberar el bloqueo.

La prueba con cinco hilos mostró la diferencia entre ambas situaciones. Sin el bloqueo podían entrar cinco citas con un cupo de una, mientras que con él solo entraba una. También se corrigió la lectura de fechas para rechazar días inexistentes. La API terminó esta fase con 88 pruebas correctas y quedó preparada para conectarla con Android.

App Android y recorrido completo · hasta el 15 de octubre de 2026

Etapa: desarrollo. **Dedicación estimada:** 16 horas.

Se completaron las pantallas de acceso, cliente y barbero con vistas XML y la separación entre vistas, modelos de vista y repositorios. Las peticiones de red se ejecutan fuera del hilo principal. En el emulador se recorrió el proceso desde la invitación hasta la confirmación de una cita, con la API y MySQL en funcionamiento.

Las pruebas de uso revelaron problemas que no aparecían en las pruebas unitarias. El teclado tapaba el código recién generado, algunos botones desactivados parecían activos y el calendario usaba el idioma del emulador. Se ajustaron el foco, los estilos y la configuración del idioma.

La alineación de las horas requirió revisar el manifiesto. Android ignoraba la alineación del texto porque el soporte de escritura de derecha a izquierda estaba desactivado. Al corregir esa opción, las etiquetas quedaron centradas. La fase terminó con 25 pruebas unitarias correctas, lint sin errores y capturas del recorrido. Los diagramas se actualizaron con las clases y métodos del código final.

Memoria y preparación de la defensa · desde el 16 de octubre de 2026

Etapa: cierre. **Dedicación estimada para la fase completa:** 18 horas, incluidos el ensayo y la grabación pendientes.

Se redactaron los capítulos de la memoria, los manuales de instalación y usuario y los anexos de diseño y pruebas. Cada descripción se comparó con la aplicación y sus capturas. Se corrigieron referencias a una vista mensual de la agenda que no existe y a estados visuales que no coincidían con las pantallas.

La presentación se montó sobre la plantilla oficial, con notas para la exposición y una demostración de respaldo. Hubo que ajustar textos que se partían o se superponían al Gantt y comprobar la numeración de figuras y tablas de la memoria.

Esta fase sigue abierta. Quedan la revisión del documento definitivo, el ensayo con cronómetro y la grabación final de la defensa. El vídeo de respaldo también debe actualizarse para incluir la cita manual añadida después.

Cita creada por el barbero · 17 de octubre de 2026

Etapa: desarrollo, ampliación P2. **Dedicación estimada:** 4 horas.

Se añadió la creación de una cita para un cliente aprobado desde la agenda del barbero. La API comprueba que la hora no haya pasado y que no exista otra cita

activa a la misma hora. La cita se guarda confirmada, sin pasar por una solicitud del cliente.

Al girar la pantalla, los desplegables solo mostraban el elemento elegido porque Android restauraba el texto y filtraba la lista. Se guardó la selección de forma explícita y se restauró sin filtrar. También se recuperó el listener del calendario tras la recreación de la pantalla.

Se añadieron nueve pruebas a la API y una a la app, con lo que los conjuntos llegaron a 97 y 26 pruebas respectivamente. La función se comprobó en el emulador y se incorporó a los manuales, requisitos y diagramas. Las demás mejoras P2 permanecen fuera del alcance terminado.

Balance del trabajo

La estimación central suma 78 horas: 6 de inicio, 14 de diseño, 40 de desarrollo y 18 de cierre. Corresponde a unas 18 horas semanales, con 3 horas por día laborable y unas 3 horas en todo el fin de semana. Una parte del cierre todavía está prevista. El registro muestra que las comprobaciones de concurrencia y el uso del emulador permitieron detectar errores que no se veían al revisar las pantallas o el esquema por separado. En un proyecto posterior convendría conectar antes una pantalla sencilla con la API y registrar los cambios en commits más pequeños.